SAQURE: AI Planning-Driven Self-Adaptation for Runtime Security and QoS of Microservice Applications

LIN CUI, Karlsruhe Institute of Technology, Germany
RUIQI GAO, University of Lugano, Switzerland
RAFFAELA MIRANDOLA, Karlsruhe Institute of Technology, Germany

The high runtime dynamism of microservice applications increases security uncertainty and reshapes the attack surface, often leading to security drift, where the design-time or static security controls rapidly lose their effectiveness during operation. Self-adaptive systems have been widely adopted to manage such uncertainty by enabling software to monitor its execution context and adapt its behavior at runtime. However, when applied to runtime security management, existing self-adaptive approaches typically rely on static assumptions and therefore struggle to assess and manage security risks as the system and its environment evolve. In particular, although the classical MAPE-K (Monitor, Analyze, Plan, Execute, Knowledge) framework supports self-adaptation in uncertain environments, existing MAPE-K-based approaches for security primarily rely on static security assessments and lack mechanisms for dynamic, environment-aware security risk assessment. In this paper, we propose SAQURE, an AI planning-driven, self-adaptive approach that enables environment-aware, near real-time security management for microservice applications. SAQURE extends the MAPE-K framework by maintaining explicit and continuously updated representations of the system configuration and adversarial environment in the Knowledge component, and by performing dynamic security assessment during analysis. This assessment employs AI planning based on the current system configuration, environmental conditions, and potential attacker behavior to generate and evaluate interpretable attack paths at runtime. Based on this analysis, SAQURE selects adaptation strategies that minimize security risk while preserving acceptable Quality of Service (QoS). We evaluate SAQURE on two representative microservice benchmarks subject to diverse runtime changes. The results show that SAQURE consistently selects adaptation options with the lowest security risk and acceptable QoS in near real time, and that it scales effectively as the number of vulnerabilities increases. To support reproducibility and future research, we release an extensible generic adversarial environment domain with approximately 300 atomic action specifications, covering attacker, application, user, and infrastructure behaviors.

CCS Concepts: • **Security and privacy → Software and application security**; • **Software and its engineering → Software creation and management**; • **Computing methodologies** → *Planning for deterministic actions.*

Additional Key Words and Phrases: Microservice applition runtime security, Self-adaptive system (SAS), Dynamic security assessment, AI planning

ACM Reference Format:
Lin Cui, Ruiqi Gao, and Raffaela Mirandola. 2026. SAQURE: AI Planning-Driven Self-Adaptation for Runtime Security and QoS of Microservice Applications. 1, 1 (September 2026), 36 pages. https://doi.org/XXXXXXX.XXXXXXX

Authors' Contact Information: Lin Cui, Karlsruhe Institute of Technology, Karlsruhe, Germany, lin.cui@kit.edu; Ruiqi Gao, University of Lugano, Lugano, Switzerland; Raffaela Mirandola, Karlsruhe Institute of Technology, Karlsruhe, Germany, raffaela.mirandola@kit.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM XXXX-XXXX/2026/9-ART
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Microservice applications decompose monolithic systems into loosely coupled, independently deployable services that encapsulate business capabilities behind well-defined interfaces, enabling independent development, deployment, updating, and scaling at runtime [11]. As a result, microservice architectures have seen broad adoption across industries: a 2023 Gartner survey reports that 74% of organizations already employ microservices, while the remaining 26% plan to adopt them [15]. However, this widespread adoption has been accompanied by escalating security challenges, as microservice applications at runtime are increasingly exposed to attacks targeting service APIs, third-party dependencies, and distributed communication channels. These threats are further amplified by the high update frequency of microservice applications at runtime (10-100 times per day [18]), driven by internal changes, such as configuration and dependency updates, as well as external environmental dynamics. This heightened runtime dynamism increases security uncertainty, reshapes attack surfaces, and accelerates *security drift* [43], where design-time or static mechanisms quickly become outdated or ineffective at runtime. For example, upgrading a microservice application's Spring Framework dependency from 5.5.31 to 6.1.3 at runtime to adopt new features or patch a bug may inadvertently introduce vulnerabilities that were previously irrelevant (e.g., CVE-2024-22243 and CVE-2024-38820, affecting Spring Framework 6.1.0–6.1.3), thereby exposing new attack surfaces. Equipping microservice-based applications in the running environment with adaptable security mechanisms that continuously adjust to fluctuating attack surfaces and evolving threats, while preserving Quality of Service (QoS), is an urgent necessity.

Traditional runtime security approaches, which primarily rely on either static, predefined rules and policies [12, 29, 39], or runtime data collection and dynamic analysis [16, 18, 27], struggle to keep pace with rapidly fluctuating attack surfaces and continuously evolving threats at runtime. These solutions face an inherent tradeoff between security effectiveness and runtime overhead in highly dynamic environments: static rule- and policy-based approaches react immediately, but become ineffective as runtime condition changes; conversely, runtime data-driven analysis improves effectiveness under change but often at the cost of substantial computational and storage overhead, which may degrade QoS. Moreover, even when enhanced with Machine Learning (ML) techniques, these approaches depend on large-scale priori labeled data (ground truth), often exhibit high false-positive rates and limited interpretability [27], hindering their practical deployment in highly dynamic environments. Self-adaptive systems (SASs) [47] have therefore emerged as a promising direction, because they combine a managed system that provides core functionality and a managing system that monitors and adapts it based on the new knowledge gathered at runtime autonomously. Existing SAS-based solutions for runtime security mainly adopt either [3]: (i) self-protection mechanisms (e.g., Moving Target Defense [22, 25, 28]) that proactively shift the attack surface at the potential cost of degraded QoS , or (ii) integrates security assessment into adaptation to identify the optimal adaptation option. A recent representative work, AQUA [3] integrates security assessment into a MAPE-K (Monitor, Analyze, Plan, Execute, Knowledge) loop. However, AQUA relies on security scores computed offline and assumes a static system and environment. As a result, such as newly exposed vulnerabilities, can invalidate its assessment, making it unable to cope with fluctuating attack surfaces. To enable effective self-adaptive runtime security for highly dynamic microservice applications, security assessment must therefore be (a) dynamic, operating on the latest system and environment state, and (b) satisfy the stringent runtime constraints. This lead to two core challenges:

– How to abstract both endogenous system configurations and exogenous adversarial environment factors of microservice applications within a model, while keeping it consistent with the latest runtime state?

– How to dynamically evaluate the security of microservice applications and select the optimal adaptation option that minimizes risk, ensures acceptable QoS based on up-to-date system and environment states, and meets strict timing requirements?

To address these challenges, we present SAQURE [1], an AI-planning-driven self-adaptive system that extends the classical MAPE-K framework for microservice-based applications. SAQURE models the endogenous system configuration and the exogenous adversarial environment of microservice applications into an abstract **Adversarial Environment Domain (AED)**, with its **Instances (AEDIs)** representing concrete snapshots of specific applications. During runtime, SAQURE incrementally updates AED/AEDIs maintained in the Knowledge module based on the latest observations collected by Monitor, keeping them consistent with the current system and environment state. To identify the optimal adaptation option while ensuring acceptable QoS, SAQURE performs AI-planning-driven **Environment-Aware Dynamic Security Assessment (EADSA)**, which generates interpretable Attack Paths (APs) and quantifies security risk by considering vulnerability severity and step-wise exploitation successful probability, accounting for both deterministic and uncertain factors. By continuously selecting and applying optimal adaptations, SAQURE dynamically adjusts the managed microservice application to evolving attack surfaces while preserving acceptable QoS. In summary, the contributions are:

– We propose an enhanced self-adaptive framework SAQURE to adapt microservice applications to runtime evolving attack surfaces, minimizing the risk while ensuring the acceptable QoS.
– We elaborate on a partially updatable AED and its Instance to formally model the dynamic microservice applications with their evolving adversarial environment.
– We design Environment-Aware Dynamic Security Assessment, using adaptive AI planning to generate interpretable APs and quantify security risk based on both deterministic and uncertainty-impacted step-wise successful probability and the exploited vulnerability's severity.
– We evaluate SAQURE on two representative microservice benchmarks, demonstrating its effectiveness in optimal adaptation option selection under diverse runtime changes and showing that it scales well with increasing numbers of vulnerabilities.
– In addition to the replication package, we publicly release an extensible, generic AED comprising approximately 300 atomic action specifications to support reuse across microservice applications.

The rest of this paper is organized as follows. Section 2 reviews related work, followed by preliminaries and the adversary model in Sections 3 and 4. Section 5 presents SAQURE's design requirements and architecture. Section 6 defines the AED/AEDIs and describes their PDDL-based specifications. Section 7 details the runtime adaptation process, including AED/AEDIs updates, EADSA, and option selection. Sections 8 and 9 describe the research questions, the implementation, and the environmental design, as well as the main results. These are followed by the discussion and conclusions in Sections 10 and 11.

2 Related Work

This section reviews two lines of related work relevant to our study: security assessment (Section 2.1) and microservice runtime security (Section 2.2).

2.1 Security Assessment

Security assessment systematically analyzes system configurations, behaviors, and operational contexts to identify vulnerabilities and evaluate the overall security posture under potential attacks. Existing approaches are generally classified into static and dynamic assessments, depending on whether the evaluation relies on real-time system state.

[1]The high-level idea behind SAQURE has been presented in [8]

Major static security assessment (SSA) methods can be broadly organized into vulnerability- & configuration-based [1, 44] and model-based [21, 24, 35] approaches. The first category relies on asset inventories and vulnerability databases, such as Common Vulnerabilities and Exposures (CVEs) [33], typically using the Common Vulnerability Scoring System (CVSS) [13] for severity scoring. While CVSS scores are widely adopted, they are fixed per CVE and do not account for differences in system configuration or operating environment. Model-based methods employ Attack Graphs (AGs), Attack Trees (ATs) [42], Attack Paths (APs) [42] and related formalisms to visualize potential attack paths that could compromise a system. Representative tools include MulVAL [35], and TVA [23]. However, these static approaches rely on time-consuming offline expert analysis and struggle to accommodate evolving system states and emerging vulnerabilities. As system scale increases, state explosion further limits their applicability to rapidly evolving conditions [31].

Dynamic security assessment (DSA) methods continuously evaluate the system state and environmental context beyond the offline stages of the application life cycle, enabling timely security decisions at runtime [6]. Most contemporary DSA models leverage AI/ML techniques, including Bayesian Network-based [7] , and AG/ATs-based [38], in order to harness heterogeneous real-time data coming from its system and environment. Artificial immune system-based methods [17] have also been applied to enhance adaptability to dynamic environments. Despite these advances, existing DSA methods face significant challenges. BN- and AT-based methods often do not support partial updates, suffer from limited scalability, low real-time performance, and low interpretability for non-expert stakeholders. NN-based methods offer faster computation but require frequent and time-consuming retraining on large-scale ground truth, are prone to model drift, and lack interpretability, raising concerns about transparency and trustworthiness. Immunity-based approaches tend to generate high false positive rates and struggle to scale to large systems.

These limitations highlight the need for dynamic, near-real-time security assessment that can accommodate rapidly changing system and environmental conditions. SAQURE addresses this need by generating interpretable attack paths at runtime from continuously updated system and environment states, enabling timely security evaluation of candidate adaptation options.

2.2 Microservice Runtime Security

Secure runtimes are essential for microservice deployments [5]. The classical runtime security approaches can be broadly categorized into two classes: (i) methods based on static rules or policies, such as access control [29], policy-based enforcement [12], and VM or container isolation [39]; and (ii) runtime data-driven methods, including anomaly detection [27], and host monitoring [16]. These approaches help mitigate runtime security threats to varying degrees. Nevertheless, frequent updates in microservice applications (10–100 times per day [18]) introduce significant *security drift* [43], as changes in system configurations and the environment can cause attack surfaces to fluctuate and render static security mechanisms ineffective. Static rule- and policy-based methods are therefore poorly suited to such dynamics, whereas runtime data-driven methods can better accommodate them through continuous monitoring and runtime analysis. However, they often incur substantial computational and storage overhead, potentially delaying detection and response to lag behind attacks and negatively affecting system performance and QoS. ML-based approaches may further suffer from limited interpretability and high false-positive rates [27], complicating their practical deployment in highly dynamic microservice environments.

Recently emerged works have explored applying the mechanisms of SASs [47], which autonomously adjust to system and environment changes, to microservice applications in order to handle risks under dynamic runtime conditions. These approaches fall into two major categories [3]: (i) self-protection approaches , which detect, analyze, and mitigate security threats by applying appropriate system reconfiguration to shift attack surfaces; and (ii) approaches that incorporate

security risk analysis into the adaptation process, assessing the security risks of potential adaptation options. Moving Target Defense (MTD) [22], which proactively and adaptively shifts the system's attack surface by modifying IP addresses, ports, service instances, and configurations at runtime, has been increasingly applied to microservice architectures as a representative self-protection mechanism (category (i)). For example, Jin et al. [25] proposed DESOM, a framework that automatically monitors updates in container-based cloud environments, evaluates the effectiveness of MTD strategies, and dynamically optimizes MTD configurations by analyzing the mediation centrality of microservice nodes in the attack graph. However, DESOM may increase service response time, negatively impacting QoS and user experience. To mitigate this limitation, another MTD-based self-protection approach, AD2S, was developed by Li et al. [28]. AD2S balances QoS and security using a deep reinforcement learning (DRL)-based optimization algorithm to dynamically adjust parameters such as the cleaning cycle and the microservice replica counts. Although these dynamic, adaptive adjustments proactively modify the attack surface and improve the adaptability to optimal security and QoS, DRL-based methods inherently require continuous retraining in fast-changing environments and suffer from weak interpretability. Beyond self-protection, RAMSES [41] implements a MAPE-K-based framework for maintaining QoS of microservice application, without considering security. AQUA [3] extends RAMSES by incorporating security assessment into the MAPE-K adaptation loop (category (ii)) to jointly ensure QoS and security of microservice applications. However, AQUA performs security assessment offline, resulting in a static security model at runtime. Thus, any system or environmental changes, such as newly exposed vulnerabilities, can immediately invalidate its assessment, making it unable to handle fluctuating attack surfaces.

In contrast, SAQURE integrates an environment-aware, dynamic security assessment into the MAPE-K adaptation loop, enabling near-real-time evaluation of candidate adaptation options. It continuously adapts microservice applications to runtime system and environmental conditions, minimizing security risk while maintaining acceptable QoS. Table 1 summarizes the key differences between SAQURE and existing self-adaptive microservice runtime security approaches.

Table 1. Comparison of SAQURE with Related Microservice Runtime Security Approaches

Approach	MAPE-K-based Framework	QoS-aware	Security-aware	Attack Modeling	Dynamic Security Assessment	Decision Interpretability
DESOM [25]	–	–	✓	✓	–	✓
AD2S [28]	–	✓	✓	✓	–	–
RAMSES [41]	✓	✓	–	–	–	✓
AQUA [3]	✓	✓	✓	–	–	✓
SAQURE	✓	✓	✓	✓	✓	✓

3 Preliminaries

This section introduces some key concepts about AI planning (Section 3.1), Discrete-Time Markov Chains (DTMCs) (Section 3.2), and the RAMSES self-adaptive framework with a managed microservices exemplar (Section 3.3).

3.1 AI Planning and PDDL

AI planning is a core area of Artificial Intelligence (AI) that studies how to automatically generate a sequence of actions, called a *plan*, to transform a system from an initial state to a desired goal state [14]. Planning problems are typically specified in a declarative manner and solved by an AI planner through systematic search. The Planning Domain Definition Language (PDDL) is a

standard language for describing AI planning problems. It standardizes problem description into two complementary files. The *domain file* defines the abstract environment model, including object types, predicates, numeric functions, and action schemas. Predicates are Boolean relations that describe whether state properties over typed objects hold. Functions represent numeric state variables (also called numeric fluents) over typed objects, whose values may change over time, e.g., representing costs or other quantitative properties. Each action schema specifies its parameters, its preconditions expressed as conjunctions of predicates and/or numeric constraints that must hold for the action to be applicable, and its effects, which describe how predicates and function values are updated after execution. The *problem file* instantiates the domain for a specific scenario by declaring concrete objects, defining the initial state as a set of grounded predicates and function assignments, and specifying one or more goal conditions to be reached from the initial state. Given a domain and a problem specification, a PDDL-based AI planner searches for a valid sequence of grounded actions that transforms the initial state into a state satisfying the goal conditions.

3.2 DTMCs

DTMCs [32] are a formalism for modeling systems with probabilistic state transitions at discrete time steps. A DTMC is defined as a tuple $\mathcal{M} = (S, s_0, P, L)$, where S is a finite set of states, $s_0 \in S$ the initial state, $P : S \times S \to [0, 1]$ the transition probability function satisfying $\sum_{s' \in S} P(s, s') = 1$ for all $s \in S$, and $L : S \to 2^{\Phi}$ assigns atomic propositions from a set Φ to states. When applied to microservice applications, states S typically represent the entry point, individual microservices, and terminal outcomes (e.g., `SUCCESS` or `FAILURE`), while transitions matrix P capture probabilistic invocation dependencies among API calls between microservices. The DTMC can be extended with a reward structure $r = (r_s, r_a)$ to quantify non-functional properties, where $r_s : S \to \mathbb{R}\geq 0$ assigns rewards to individual states, reflecting, for instance, the cost or benefit of reaching a particular system state, and $r_a : S \times S \to \mathbb{R}\geq 0$ assigns rewards to transitions, representing metrics such as execution time, probability-weighted cost, or risk associated with invoking a specific API call. DTMC models can be constructed manually or inferred from execution traces, with transition probabilities estimated from empirical measurements of benchmark availability, and can be continuously updated at runtime via the MAPE-K loop [3].

3.3 Self-adaptive Framework: RAMSES

We adopt an architecture-based adaptation approach [47], where a self-adaptive system (SAS) consists of a managed system providing core functionality and a managing system that monitors and adapts the parts of the managed system to achieve adaptation goals autonomously at runtime, using knowledge collected at the runtime.

RAMSES [40] is an autonomic framework for engineering customizable microservice-based SASs. It implements the MAPE-K control loop to enforce user-defined QoS and self- properties, using runtime models and configuration state in its Knowledge service to support adaptation decisions. As shown in Figure 1, RAMSES comprises two main components: (i) a microservice-based managed application exemplar that operates in the running environment to provide domain functionality to users, and (ii) a managing system consisting of five independent microservices that implement the MAPE-K loop to monitor and adapt the managed system with minimal human intervention.

Managed microservices exemplar: SEFA. SErvice-based E-Food Application (SEFA) [40] serves as a managed system in RAMSES. Implemented with Spring Boot and Spring Cloud, SEFA is a representative microservice benchmark that allows users to browse menus and place food orders with credit-card payments. It consists of a web front-end and four backend microservices: `ordering` (cart and order management), `payment proxy` (third-party payment integration), `delivery proxy` (food delivery coordination), and `restaurant` (menu and restaurant management).

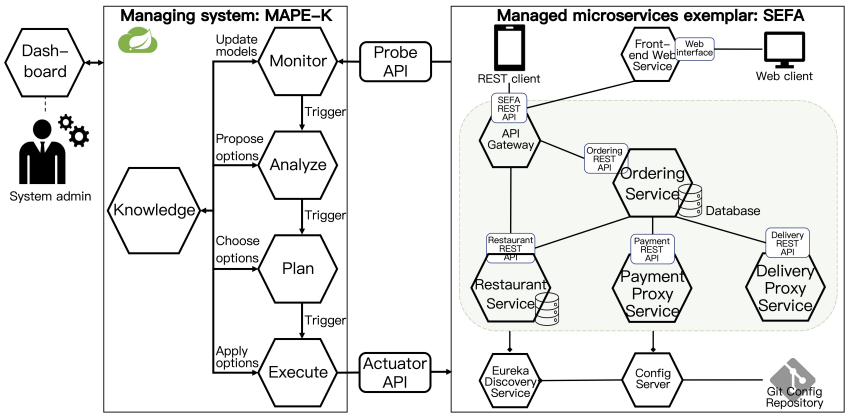


Fig. 1. Overview of the RAMSES architecture with SEFA as the managed microservice exemplar.

Managing system: MAPE-K loop. The managing system in RAMSES follows the MAPE-K paradigm: the Knowledge service maintains the data required in the self-adapt loop, such as quality goals, the runtime model (including QoS metrics, e.g., availability, response time, configuration parameters, alternative implementations and operational profiles). Monitor collects runtime data from the managed system, and records snapshots with metrics (CPU usage, HTTP request stats, etc.) into Knowledge. Analyzer updates QoS indicators from these snapshots and generates adaptation options (instances scaling, implementations switching, load-balancer weight adjustment) for each service when adaptation goals are unmet. Plan selects the optimal adaptation option, and execute applies it to the managed system. Through this design, RAMSES ensures autonomous, near-real-time adaptation while maintaining user-specified QoS objectives and self-* properties.

4 Adversary Model

We assume trustworthy service providers and external attackers who may attempt to exploit vulnerabilities of microservices during runtime to compromise the target application. The adversary's goals, attack stages, and capabilities are described below.

Attack Goals. The attacker's objectives follow the STRIDE threat categories [42]: (i) *Spoofing*, impersonating authenticated users or services; (ii) *Tampering*, modifying application data or system state without authorization; (iii) *Repudiation*, denying having performed actions; (iv) *Information Disclosure*, obtaining sensitive information without permission; (v) *Denial of Service*, disrupting service availability or operation; (vi) *Elevation of privilege*, escalating privileges beyond those initially available to the attacker, such as executing arbitrary code within the target application.

Attack stages & attacker capabilities. We assume external attackers who initially have no access to protected system internals. The attackers possess no prior knowledge of or access to non-public system internals, such as internal configurations, credentials, or cryptographic keys, unless these become available as a consequence of an attack step. Their computational resources are moderate-sufficient, for example, to brute-force weakly hashed credentials but inadequate to break strong cryptography. Attackers are assumed to have knowledge of publicly disclosed vulnerabilities, such as CVEs, and publicly available exploitation TTPs(Tactics, Techniques, Procedures). In our model, such exposure is specified as a dedicated application vulnerability exposure action associated with the corresponding CVE-ID in the AED (see Section 6.2). A vulnerability exploitation can be initiated only when the application exposes attack surfaces associated with known vulnerabilities under the certain system and environmental state. At runtime, changes in the managed system's

configurations (e.g., dependencies, versions, and security settings) and environmental conditions (e.g., user behavior) may alter these states and, consequently, vulnerability exposure.

Each STRIDE-category attack is assumed to arise from a single vulnerability exploitation rather than combined exploits. We use the Cyber Kill Chain [20] as a reference for organizing attack actions into five stages: *Exposure*, *Weaponization*, *Delivery*, *Exploitation*, and *Impact*. Table 2 summarizes the representative attacker capabilities associated with each stage.

Table 2. Representative Attacker Capabilities across Attack Stages

Stage	Representative Attacker Capability
Exposure	Discovers or confirms exposed services and relevant system characteristics, including vulnerable components and applicable exploitation techniques.
Weaponization	Crafts malicious artifacts or triggers, such as scripts, files, or payloads, targeting known service vulnerabilities.
Delivery	Delivers the malicious artifact or trigger through an applicable channel, such as a URL, file, or HTTP(S) request.
Exploitation	Triggers the vulnerable behavior or logic through one or more exploitation actions.
Impact	Achieves the intended STRIDE security goal.

5 Framework Overview

This section first derives the key design requirements (Section 5.1), and then presents the SAQURE architecture (Section 5.2) as a systematic solution that satisfies these requirements.

5.1 Design Requirements

In order to enable effective self-adaptation via integrated security assessment for microservice applications, minimizing runtime risk while ensuring QoS, SAQURE is designed to satisfy the following requirements:

R1: Modeling & runtime updates. Security assessment and self-adaptation relies on up-to-date system and environment information maintained in the Knowledge module to support adaptation decisions. Accordingly, the framework must: (**A**) comprehensively model the managed system and its external environment, capturing key entities, their attributes, states, relationships, and actions; and (**B**) support incremental model updates at runtime to consistently reflect both endogenous system changes and exogenous environmental dynamics.

R2: Runtime security assessment & adaptation decision-making. Based on the latest system and environment information, the framework must: (**A**) perform near real-time security risk analysis and dynamic assessment; and (**B**) select the optimal adaptation option that minimizes security risk while ensuring acceptable QoS.

5.2 SAQURE Architecture

To minimize the security risk while maintaining acceptable QoS, we propose SAQURE, an extension of the classical MAPE-K self-adaptive loop. SAQURE introduces two key enhancements: (i) the Knowledge module maintains a PDDL-specified AED and AEDIs, representing both the managed system and its environment; and (ii) the Analyzer integrates an AI planning-driven EADSA method to evaluate and guide adaptation decisions. Figure 2 shows the SAQURE framework, it contains two primary stages:

Offline stage. Prior to deployment, the managed application and its environment are formally modeled and specified to be pre-stored within the Knowledge module. Specifically, leveraging the standardized PDDL modeling formalism for capturing system dynamics [14], the managed application's endogenous configuration and exogenous environment are represented as the AED and AEDIs. The AED specifies a general schema encompassing entity types, states, relationships,

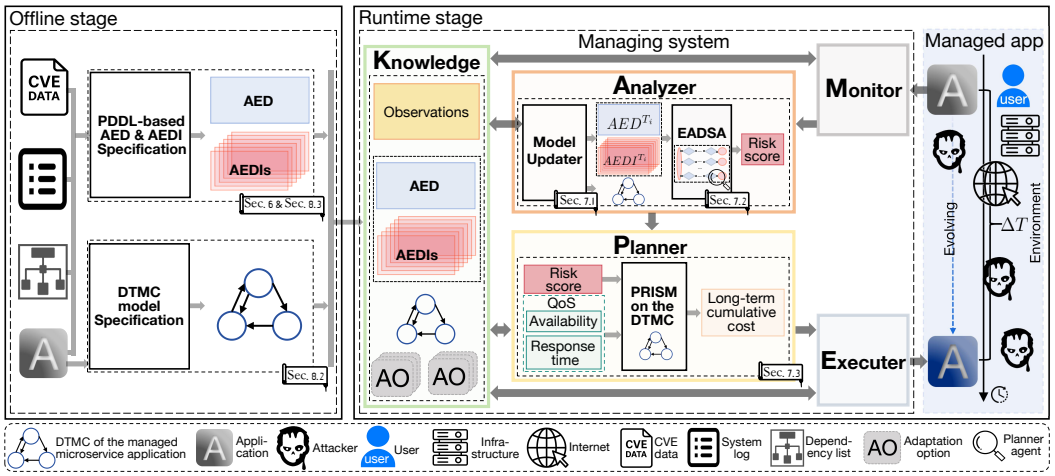


Fig. 2. The framework of SAQURE.

attributes, and actions. Each AEDI instantiates these schema elements with concrete objects and values to represent the application's state at a given time point and define goal states for runtime risk assessment (**R1.A**). The initial AED and AEDI specifications are encoded in PDDL 2.1 syntax [14] with the system and environment's initial settings and values, and are incrementally updated at runtime (Section 6 describes the formal modeling, and Section 8.3 details implementation). Additionally, following AQUA [3], we derive a runtime DTMC model of the managed microservice application by analyzing its inter-service dependencies to represent its probabilistic execution behavior (Section 8.2). Once specified, the initial AED/AEDIs and the DTMC model are pre-stored in the Knowledge module and updated at runtime to support adaptation decision-making.

Runtime management. During runtime, SAQURE's MAPE-K feedback loop continuously evaluates and selects optimal implementation options aimed at minimizing security risk while preserving acceptable QoS. Specifically, the runtime operation is structured as follows:

K: To accommodate the highly dynamic nature of the microservice application and its evolving exogenous environment, the Knowledge module maintains the AED/AEDIs and DTMC model with the latest observations. The initial formal specifications of AED and AEDI, established offline by using PDDL, are partially updated at runtime to reflect changes in both the endogenous system and the exogenous environment. Concurrently, the transition probabilities of the DTMC model are also synchronized.

M: To support maintaining up-to-date knowledge and runtime analysis, the Monitor periodically and asynchronously collects the required runtime telemetry (e.g., system configuration snapshots, runtime metrics, security-related settings) and external threat intelligence (TI).

A: Based on the latest observations, the Analyzer updates the AED or corresponding AEDIs through partial updates (**R1.B**, Section 7.1). To support runtime self-adaptation aimed at risk minimization, it invokes an AI planning-driven Environment-Aware Dynamic Security Assessment (EADSA) method to dynamically compute security risk scores for available adaptation options in near real-time using the most recent AED and AEDIs (**R2.A**, Section 7.2). Once the risk scores for candidate adaptation options are computed, the Analyzer records each option along with its associated risk score and QoS estimates in the Knowledge module.

P: When invoked by the Analyzer, the Planner retrieves implementation options and associated metrics from the Knowledge module. It then evaluates alternatives using the PRISM probabilistic model checker [26] on the managed application's DTMC model to simulate the application's execution over a finite horizon. A long-term cumulative cost is accumulated to quantify each option's effectiveness, the option with minimal expected cost is selected (**R2.B**, Section 7.3).

E: Once the decision is made, the Executor applies the selected adaptation to the running app and feeds execution outcomes back to the Knowledge module to complete the feedback loop.

6 Modeling of Microservice App and Adversarial Environment (R1.A)

This section introduces the AED and AEDI to formally model the microservice application and the adversarial environment (Section 6.1), and then presents their specification using PDDL (Section 6.2).

6.1 Adversarial Environment Domain and its Instance

6.1.1 AED. We formally model the microservice application and the adversarial environment at a given time point T_i as an abstract schema, called AED. It captures the typed entities, their states, relationships, attributes, and the possible actions within the system or exogenous environment that may affect system security.

Definition 1. *The **Adversarial Environment Domain (AED)** at T_i is defined as a quadruple $AED^{T_i} = (\mathcal{T}, \mathcal{P}, \mathcal{F}, \mathcal{A})$, where $\mathcal{T} = \{\tau_i\}$ is the set of entity types, $\mathcal{P} = \{p_i\}$ is the set of predicates representing states, relationships or non-numeric attributes of typed entities, $\mathcal{F} = \{f_i\}$ is the set of numeric functions representing numerical attributes of typed entities, and $\mathcal{A} = \{a_i\}$ is the set of atomic actions over typed entities.*

Each entity type $\tau_k \in \mathcal{T}$ defines a class of objects in the microservice application or its external adversarial environment. It serves as a type system, constraining and organizing the arguments of predicates $\mathcal{P}$, functions $\mathcal{F}$, and actions $\mathcal{A}$ by classifying entities into distinct semantic categories. AED includes four main entity types: ***application***, ***attacker***, ***user***, and ***infrastructure***, as well as other security-relevant types , such as *cve-identifier*, *exploit-technique*, *browser*, *http-post-request*, *endpoint*, *payload*, *server*, *framework*, *website*, *file*, *java-library*, *credential*.

Each predicate $p_i \in \mathcal{P}$ encodes a boolean condition over entities, indicating whether a particular state, relationship or non-numeric attribute holds (*true*) or not (*false*).

Example 6.1. Typical predicates include "*running-browser*", representing the "running" state of a *browser*-typed entity, and "*has-library*", representing the "has" relationship between an *application*-typed entity and a *library*-typed entity, among others.

Each function $f_i \in \mathcal{F}$ captures numeric attributes by mapping typed entities to a numeric value.

Example 6.2. Typical functions include *version(·)*, returning the version number of a given entity, and *port(·)*, returning the open port number of a given entity, among others.

Each atomic action $a_i \in \mathcal{A}$ describes state transitions caused by entities of type *application*, *attacker*, *user*, or *infrastructure*. It is defined as a quadruple $a_i = \{N_i, Para_i, Pre_i, Eff_i\}$, where N_i is a name identifier interpretable in natural language. $Para_i$ is the set of typed parameters specifying the entities involved in the action execution. The states, relationships, attributes of these entities may change upon executing the action. The precondition $Pre_i \subseteq \mathcal{P} \cup \mathcal{F}$ consists of *controllable deterministic conditions* (the conditions that the adaptation engine can observe and modify), represented by the combination of predicates p_i or numeric functions f_i, that must hold for the action to be applicable. The effect $Eff_i \subseteq \mathcal{P} \cup \mathcal{F}$ describes the resulting state, relationships, attributes after executing the action. Depending on the semantic type of the action's initiator, there

are four main types of actions in AED involved in the attack process: ***application action***, ***attacker action***, ***user action***, and ***infrastructure action***.

Example 6.3. Table 3 presents a representative example of each *application action*, *attacker action*, *user action*, and *infrastructure action*, in order from top to bottom. ① The *application action* example indicates that one *application* receives a *http post request* with a malicious *payload* through a *receive endpoint*. ② The *attacker action* example indicates that one *attacker* sets up a new phishing *website* on a *controlled server* to exploit a *CVE* to compromise the target *application*. ③ The *user action* example indicates that one *user* reads a *message* on a *communication channel*, like email, etc. ④ The *infrastructure action* example indicates that one poisoned *dns-server* serves an *application*'s query for a *legitimate server* with a *malicious ip address*.

Table 3. Examples of Four Action Types and Their Semantics, Name, Parameters, Preconditions, and Effects

Name N_i	**Parameters** $Para_i$	**Precondition** Pre_i	**Effect** Eff_i
① "***application***-*receives-http-post-request-with-malicious-payload*"	*application*-typed entity (A), *http-post-request*-typed entity (HP), *payload*-typed entity (P), *receive-endpoint*-typed entity (RE)	1. HP "contains" P; 2. HP is in "sent" state; 3. Malicious P is in "transmitted" state; 4. A "has" RE	1. HP is in "received" state
② "***attacker***-*forge-phishing-website-mimic-legitimate-app*"	*website*-typed entity (W), *controlled-server*-typed entity (S), *CVE*-typed entity (C), *application*-typed entity (A)	1. A "exposed attack surface" due to has C ; 2. C is "exploited by" "building phishing-site"	1. W is "hosted-on" S; 2. W is in the "forged-website" state; 3. W "mimics" A
③ "***user***-*reads-message*"	*user*-typed entity (U), *message*-typed entity (M), *comm. channel*-typed entity (C)	1. U "use" C; 2. C is in "running" state; 3. C "contains" M	1. M is in the "opened" state
④ "*poisoned-****dns-infrastructure***-*returns-attacker-server-ip*"	*dns-server*-typed entity (D), *application*-typed entity (A), *legitimate-server*-typed entity (L), *malicious-ip-address*-typed entity (M)	1. D "dns poisoned" with the queried L pointed to M; 2. A "send dns query" to D	1. M as the resolution result for the L is "returned" by D; 2. L is "resolved" to M

6.1.2 AEDI. Microservice applications and their operational environments are highly dynamic, with system configurations and environmental states continuously changing over time. While *AED* provides a reusable formal schema at a reference time T_i, snapshots of a concrete system and its environment at different time points can be considered distinct instances instantiated from the schema. To represent and reason about these concrete snapshots, we introduce AEDI. At each time point T_i, $AEDI^{T_i}$ captures a concrete snapshot of the system and environment, including objects of different entity types, their states, relations, and attributes, instantiated from the associated AED^{T_i}. This enables AED to provide a unified interface for downstream dynamic security assessment.

Definition 2. *The* ***Adversarial Environment Domain Instance (AEDI)*** *at time T_i is defined as a 5-tuple $AEDI^{T_i} = (\mathcal{D}, \mathcal{O}, \mathcal{IS}, \mathcal{GS}, \mathcal{M})$, where $\mathcal{D}$ is the associated domain AED^{T_i}, $\mathcal{O}$ is the set of concrete objects at T_i, $\mathcal{IS}$ is the set of initial states, relations, and attributes of objects in $\mathcal{O}$, $\mathcal{GS}$ is the set of desired goal states, relations, or attributes of objects in $\mathcal{O}$, and $\mathcal{M}$ is the set of evaluation metrics.*

Specifically, each object $o_i \in \mathcal{O}$ instantiates a type $\tau_i \in \mathcal{T}$ of the associated domain AED^{T_i}, representing a concrete entity in the managed system or its environment at time T_i.

Example 6.4. In SEFA, at time T_i, several concrete objects instantiate entity types defined in AED^{T_i}. Specifically, "*SEFA*" is an *application* instance; "*attacker1*" is an *attacker* instance, "*user1*" is a *user* instance; "*vulnerable-dns-server1*" is a *dns-server* (*dns-server* is a subtype of *infrastructure*) instance; "*CVE_2024_38820*" is a *cve-identifier* instance; and "*browser-chrome*" is a *browser* instance.

The set $\mathcal{IS}$ selectively grounds a subset of predicates $\mathcal{P}$ and numeric functions $\mathcal{F}$ with objects in $\mathcal{O}$, capturing only those states, relations, attributes necessary to represent the initial snapshot of the system and its environment at T_i.

Example 6.5. In SEFA, at time T_i, the predicate "*two-factor-auth-enabled*" is grounded with the object "*SEFA*" to express that "*SEFA*" enables two factor authentication. The predicate "*has-library*" is grounded with the objects "*SEFA*" and "*jettison-library_SEFA*" to indicate that "*SEFA*" "has" the "*jettison-library_SEFA*". The function *version(·)* is grounded with the object "*jettison-library_SEFA*" to indicate that its version is 1.5.1-0.

Each $gs_i \in \mathcal{GS}$ is specified over grounded predicates or functions, defining security-relevant target, such as denial-of-service. $\mathcal{GS}$ and $\mathcal{M}$ serve as placeholders and will be accordingly specified to coordinate the downstream dynamic security assessment process (Section 7.2).

6.2 PDDL-based Specification

We adopt PDDL 2.1 [14] syntax to encode $AED^{T_i} = (\mathcal{T}, \mathcal{P}, \mathcal{F}, \mathcal{A})$ and $AEDI^{T_i} = (\mathcal{D}, \mathcal{O}, \mathcal{IS}, \mathcal{GS}, \mathcal{M})$ into clearly separated PDDL *domain* and *problem* files, respectively. In particular, for each CVE, we specify an *application vulnerability exposure action* to represent the reconnaissance-to-exposure transition and mark the entry point of potential attacks within the adversarial environment.

```
(define (domain AED)
(:requirements :equality :typing :strips :disjunctive-preconditions :adl)
(:types
application attacker user infrastructur dns-server - infrastructure cve-identifier
exploit-technique browser http-post-request receive-endpoint - endpoint payload
controlled-server - server framework website file java-library credential ... )
(:predicates
(spoofing ?Application - application) ...
(elevation-of-privilege ?Application - application) ;;Goal is defined in Section 7.2
(exploit-by ?CVEID - cve-identifier ?ExploitTechnique - exploit-technique)
(running-browser ?Browser - browser)
(has-library ?Application - application ?Library - java-library)
(two-factor-auth-enabled ?Application - application) ... )
(:functions
(version ?Software) (port ?Port) ... (total-cost) )
(:action application-receives-http-post-request-with-malicious-payload
 :parameters (?Application - application ?HTTPRequest - http-post-request
              ?MaliciousPayload - payload ?ReceiveEndpoint - receive-endpoint)
 :precondition (and (http-post-request-sent ?HTTPRequest)
                    (contains ?HTTPRequest ?MaliciousPayload)
                    (malicious-payload-transmitted ?MaliciousPayload)
                    (has-receive-endpoint ?Application ?ReceiveEndpoint))
 :effect (and (http-post-request-received ?Application ?HTTPRequest)
              (increase (total-cost) 1)) ;;Action cost is detailed in Section 7.2
```

application action

```
(:action attacker-forge-phishing-website-mimic-legitimate-app
 :parameters (?Application - application ?PhishingSite - website
              ?ControlledServer - controlled-server ?CVEID - cve-identifier)
 :precondition (and (exposed-attack-surface ?Application ?CVEID)
                    (exploit-by ?CVEID build-phishingsite) )
 :effect (and (forged-website ?PhishingSite)
              (website-hosted-on-server ?PhishingSite ?ControlledServer)
              (website-mimics-legitimate-app ?PhishingSite ?Application)
              (increase (total-cost) 1)))
```

attacker action

```
(:action user-reads-message
 :parameters (?User - user ?Comchannel - comchannel ?Message - message)
 :precondition (and (user-use-communication-channel ?User ?Comchannel)
                    (running-communication-channel ?Comchannel)
                    (com-channel-msg ?Comchannel ?Message) )
 :effect (and (msg-opened ?Message) (increase (total-cost) 1)))
```

user action

```
(:action poisoned-dns-infrastructure-returns-attacker-server-ip
 :parameters (?Application - application ?DNSServer - dns-server
              ?LegitimateServer - legitimate-server
              ?MaliciousAddress - malicious-ip-address)
 :precondition (and (dns-query-sent ?Application ?DNSServer ?LegitimateServer)
                    (dns-poisoned ?DNSServer ?LegitimateServer ?MaliciousAddress))
 :effect (and (ip-address-resolved ?LegitimateServer ?MaliciousAddress)
              (dns-response -returned ?DNSServer ?LegitimateServer ?MaliciousAddress)
              (increase (total-cost) 1))) ... )
```

infrastructure action

Fig. 3. Part of the representative AED PDDL 2.1 specifications (four colors for four types of action).

```
(define (problem AEDI)
(:objects
SEFA - application   attacker1 - attacker
user1 - user   vulnerable-dns-server1 - dns-server
http-post-request1 - http-post-request
receive-endpoint1_SEFA - receive-endpoint   payload1 - payload
browser-chrome - browser   browser-safari - browser
controlled-server1 - controlled-server   phishing_site1 - website
jettison-library_SEFA - java-library   user1_password1 - password ... )
(:init
(has-library SEFA janino-library_SEFA)
;;(1.5.1.0 → 1 × 10^9 + 5 × 10^6 + 1 × 10^3 + 0)
(= (version jettison-library_SEFA) 1005001000)
(two-factor-auth-enabled SEFA) ... )
(:goal
(and (spoofing SEFA) ... ) )
;;(:metric ... ) ;;Goal and metric is defined in Section 7.2 )
```

Fig. 4. Part of the representative AEDI PDDL 2.1 specifications (four colors for four types of concrete objects).

In specific, Figure 3 and Figure 4 show representative portions of the PDDL domain file (encoding AED) and problem file (encoding AEDI), respectively. The entity type set $\mathcal{T}$ is specified in the `(:types)` construct. $\mathcal{P}$ is specified in the `(:predicates)`. $\mathcal{F}$ is specified in the `(:functions)` (the utility of `(total-cost)` function for downstream security assessment is detailed in Section 7.2). $\mathcal{A}$ is specified using `(:action)` construct. $\mathcal{T}$, $\mathcal{P}$, $\mathcal{F}$, and $\mathcal{A}$ are encoded in a interpretable format, as illustrated in Figure 3. Once the AED^{T_i} schema is specified in the domain file, a concrete system snapshot at time T_i can then be instantiated in a problem file as $AEDI^{T_i}$. The concrete object set $\mathcal{O}$ is declared in the `(:objects)`, while the states, relations, and attributes $\mathcal{IS}$ are grounded in the `(:init)`. $\mathcal{GS}$ are specified in the `(:goal)` , and $\mathcal{M}$ are expressed via the `(:metric)`.

Application vulnerability exposure action. For microservice applications operating in the adversarial environment, the reconnaissance activity prior to actual vulnerability exploitation (e.g., port scanning or version fingerprinting) can reveal that an application exposes attack surfaces to potential attackers. To capture this notion in a declarative and PDDL-compatible way, we introduce a dedicated application exposure action for each CVE, named "`application-exposes-attack-surfaces-to-attackers-<CVE-ID>`". This action abstracts the reconnaissance-to-exposure transition and serves as the entry point for exploiting the corresponding vulnerability. Figure 5 illustrates an example application exposure action specification for CVE-2024-22243 in the AED.

```
(:action application-exposes-attack-surface-to-attackers-CVE-2024-22243
 :parameters (?Application - application ?SpringFramework- spring-framework ... )
 :precondition (and
                ① (has-dependency-on ?Application ?SpringFramework)
                   (or ;;5.3.0.0 <= version <= 5.3.31.0 or ...
                     (and
                       (>= (version ?SpringFramework) 5003000000)
                       (<= (version ?SpringFramework) 5003031000))
                     (and ... ) (and ... )) ...
                ② (url-redirect-enabled ?Application) ...
                ③ (not (WAF-malicious-request-bloack ?Application)) ... )
 :effect (and (exposes-attack-surface ?Application CVE_2024_22243) ❶
              (exploit-by CVE_2024_22243 build-phishingsite)       ❷
              (vulnerable-spring-framework ?SpringFramework)       ❸
              (increase (total-cost) 21) ;;Detailed in Section 7.2 ❹
... ))
```

Fig. 5. Application vulnerability exposure action.

The *Pre* `(:precondition)` of an exposure action encodes only decidable and observable conditions (states, relations, attributes) that characterize vulnerability exposure, including ① technical composition, such as vulnerable dependencies and architectural components (e.g., "an app depends on a vulnerable Spring Framework is specified" as"`(has-dependency-on ?Application ?SpringFramework)`" and "`(⊗ (version ?SpringFramework) <`$N_{version}$`>)", where ⊗ ∈ {=, ≤, ≥}, <`$N_{version}$`> = "major.minor.patch-milestone" is encoded via` $major \cdot 10^9 + minor \cdot 10^6 + patch \cdot 10^3 + milestone$), ② insecure permissions or access paths, such as accessible endpoints, and exposed APIs (e.g., "`(url-redirect-enabled ?Application)`"), and ③ missing or misconfigured security controls (e.g., "`(not (WAF-malicious-request-block ?Application))`"). These three categories capture most vulnerability exposures in practice [37]. These conditions are grounded from the concrete system snapshot in the `(:init)` of an AEDI. When the instantiated predicates and functions satisfy the precondition, the corresponding exposure action becomes applicable.

The *Eff* `(:effect)` of the exposure action ❶ marks the application as exposing a CVE via `(exposes-attack-surface ?Application <CVE-ID>)`, ❷ when applicable, marks the primary exploitation technique(s) for the CVE via `(exploit-by <CVE-ID> <Exploit-Technique>)` (e.g., "`(exploit-by CVE_2024_22243 build-phishingsite)`") and ❸ the vulnerable element(s) affected by the vulnerability (e.g., "`(vulnerable-spring-framework ?SpringFramework)`"), and

❹ specifies the action cost via `(increase (total-cost)` $cost(a_0)$`)`. The first three predicates can be subsequently included in the `(:precondition)` of other relevant actions' specification.

After specifying the initial version offline, both the domain file encoding the AED and the problem files encoding corresponding AEDIs are stored in the Knowledge module (the offline implementation for initializing AED/AEDIs is described in Section 8.3). These specifications provide an unified interface for downstream security assessment. While both AED and AEDI are dynamic representations evolving as runtime changes in system configuration or environmental observations, the mechanism for the dynamic update is described in Section 7.1.

7 Runtime Security and QoS Management

This section details SAQURE's runtime security and QoS management, including (i) runtime updates of the AED and AEDI (Sections 7.1), (ii) EADSA performed by the Analyzer (Section 7.2), and (iii) the selection of optimal adaptation options in the Planner that minimize security risk while ensuring acceptable QoS (Section 7.3).

7.1 AED and AEDI Updates (R1.B)

The dynamic nature of microservice applications requires that both the AED and AEDI in the Knowledge module evolve in response to changes in endogenous application configurations and exogenous environments. Upon newly observed changes in the managed system or its environment, SAQURE supports mapping these changes to AED and AEDI specifications through file-level updates. For each detected change, the Analyzer conceptually determines how such changes should be reflected in the specifications by answering: *whether the change affects* $AED^{T_{i-1}}$ *or* $AEDI^{T_{i-1}}$*? which PDDL construct? which specific item?* and *how it should be modified?* As summarized in Table 4, real-world changes are categorized as additions, deletions, or updates, and are systematically mapped to corresponding constructs in the AED domain file (F_d) or the AEDI problem file (F_p). The Analyzer then realizes these modifications through file-level updates to the corresponding PDDL specifications.

Example 7.1. Table 5 illustrates representative runtime changes and their corresponding updates to the `(:init)` construct of an AEDI problem file.

Table 4. Mapping of Real-world Changes to Constructs in AED (F_d) or AEDI (F_p)

Real-world Change		Maps Changes to the Specific Construct of F_d or F_p	
Type	**Subtype of change**	$F_{d/p}$	**Corresponding update**
Add.	New entity (of an existing type)	F_p	Adds the instantiation with its type declarations to the `(:objects)`
	New entity type	$F_{d\&p}$	Adds the type specification to `(:types)` and instantiates it in `(:objects)`
	New entity state/relation/non-numeric attribute	F_d	Adds the predicate specification under `(:predicates)`
	New numerical attribute	F_d	Adds the function specification under `(:functions)`
	New action	F_d	Adds a corresponding `(:action)` specification
Del.	Inactive or removed entity	F_p	Comments out the corresponding specification in `(:objects)`
	Entity state/relation/non-numeric attribute no longer holds	F_p	Comments out the corresponding specification in `(:init)`
	Numerical attribute removed from an entity	F_p	Comments out the corresponding specification in `(:init)`
	Numerical attribute no longer exists	F_d	Comments out the corresponding specification in `(:functions)`
	Unavailable action	F_d	Comments out the corresponding specification in `(:action)`
Upd.	The value of an entity's numeric attribute	F_p	Updates the relevant function specification in `(:init)` with the new value
	An action's precondition or effect	F_d	Updates `(:precondition)` or `(:effect)` of the corresponding `(:action)`

Based on the updated AED^{T_i} and $AEDI^{T_i}$ (with the latest timestamp T_i), the Analyzer then performs dynamic security assessment to compute runtime risk scores and guide the adaptation.

7.2 Environment-Aware Dynamic Security Assessment (R2.A)

In SAQURE, adaptation is primarily achieved by changing individual microservice implementations, which results in updated system configurations. To assess the security risk of an implementation at

Table 5. Examples of Updates in the (:init) Construct of AEDI (modifications in bold and underlined)

T	State, Relation, or Attribute	Corresponding Specification in (:init) Construct
T_{i-1}	SEFA is monitored by IDS	(IDS_monitoring SEFA)
T_i	SEFA isn't monitored by IDS	**;** (IDS_monitoring SEFA)
T_{i-1}	browser-chrome is running	(running-browser browser-chrome)
T_i	browser-chrome isn't running	**;** (running-browser browser-chrome)
T_{i-1}	SEFA depends on logbacklib	(has-library SEFA logbacklib_SEFA)
T_i	SEFA doesn't depend on logbacklib	**;** (has-library SEFA logbacklib_SEFA)
T_{i-1}	The version of logbacklib is 1.4.11-0	(= (version logbacklib_SEFA) 1004011000)
T_i	The version of logbacklib upgrades to 1.6.11-1	(= (version logbacklib_SEFA) **1006011001**)

runtime, we combine the attack vector visualization and risk assessment capabilities of Attack Path (AP) [42] with adaptive, real-time planning capabilities of AI planners when provided with updated inputs. Specifically, we formulate the AP generation as an *Environment-Aware Attack Path Planning Problem* (p_{AP}) based on AED and AEDI. When a security assessment is triggered, the Analyzer module dynamically invokes PDDL planners (i.e., AI planners that accept domain and problem files in PDDL) to generate risk-maximizing APs from the latest AED and the corresponding AEDI instantiated for the target implementation. The generated AP is then used for risk quantification.

Definition 3. ***The Environment-Aware Attack Path Planning Problem*** *is defined as* $p_{AP} : \{AED^{T_i}, AEDI^{T_i}\} \mapsto AP^{T_i}$. *Given the* $AED^{T_i} = (\mathcal{T}, \mathcal{P}, \mathcal{F}, \mathcal{A})$ *and* $AEDI^{T_i} = (\mathcal{D}, \mathcal{O}, \mathcal{IS}, \mathcal{GS}, \mathcal{M})$ *at time* T_i, *the attack path planning problem aims to generate a step-wise ordered sequence of atomic grounded actions* $AP^{T_i} = \langle a_0, a_1, \ldots, a_n \rangle$ *that optimizes the* $\mathcal{M}$ *and transforms* $\mathcal{IS}$ *into* $\mathcal{GS}$.

Each action a_i in AP^{T_i} is grounded with concrete objects from $\mathcal{O}$ of $AEDI^{T_i}$. As each composing a_i is encoded in an interpretable form in the (:action) construct of AED^{T_i}, the generated AP^{T_i} is likewise interpretable. Our approach generates *AP*s that maximize risk, a common practice in security assessment for identifying worst-case attack scenarios [2], and uses the resulting scores to quantify security near real-time. To meet strict runtime requirements, we decompose AP planning into six independent, parallelizable Environment-Aware Attack Path Planning Problems: p_{AP_S} (Spoofing), p_{AP_T} (Tampering), p_{AP_R} (Repudiation), p_{AP_I} (Information Disclosure), p_{AP_D} (Denial of Service), and p_{AP_E} (Elevation of Privilege), each aligned with a STRIDE threat objective. At runtime, when a security assessment is triggered, the Analyzer invokes planners in parallel, each solving one $p_{AP_g}, g \in \{S, T, R, I, D, E\}$ by generating an atomic action sequence $\langle a_0, a_1, \cdots, a_n \rangle$ with maximum risk that transforms $\mathcal{IS}$ into $\mathcal{GS}$. The overall risk score is then calculated from these sequences.

Computation of AP's risk score. Consistent with our adversary model, each attack path is assumed to exploit a single vulnerability. For an attack path $AP^{T_i} = \langle a_0, a_1, \cdots, a_i \rangle$, its risk depends not only on the severity of the exploited vulnerabilities but also on whether the AP can be executed successfully on the target system to cause real damage. The question then arises: *how can we quantify the overall success probability of an AP?* One intuitive approach is to decompose it into finer-grained parts and analyze the success probability of each action. Since an *AP* is a step-by-step atomic action sequence, its overall success depends on the successful execution of every preceding action a_i. Therefore, the risk score of *AP* is measured as follows:

$$RS_{AP} = \left(\prod_{i=1}^{n} P_{a_i}\right) \cdot S_v \tag{1}$$

where the severity of the exploited vulnerability S_v is quantified using its CVSS v3.1 score [13]. $P_{a_i} \in [0, 1]$ is the successful probability of each action a_i in the sequence $\langle a_0, a_1, \cdots, a_n \rangle$.

Computation of action's successful probability. In real-world scenarios, the successful probability of each action depends on both deterministic and uncertain factors. Deterministic factors are controllable or creatable conditions. For example, opening a browser ensures that the state "running"

holds for the browser; using a specific library during development creates a dependency between the library and the application. Uncertain factors, although influenced by the actor's endogenous characteristics and the exogenous environment, are not directly controllable. For instance, a user's security awareness cannot be set via a dashboard, although it may be gradually shaped through training. For each action a_i, the combination of controllable states, relationships, and attributes defined in its preconditions Pre_i represents the necessary deterministic factors. Without them, the action fails. However, even when these preconditions are satisfied, the action's success remains subject to uncertain factors. Therefore, we define the success probability P_{a_i} of action a_i as:

$$P_{a_i} = P_{Pre_i} \cdot P_{uc_i} \tag{2}$$

where P_{Pre_i} denotes the probability that action a_i's preconditions Pre_i are satisfied. If any predicate p_i or function f_i in $Pre_i \subseteq \mathcal{P} \cup \mathcal{F}$ is not satisfied for its corresponding instance due to runtime changes in system configuration or environmental conditions, or as a result of an adaptation, then $P_{Pre_i} = 0$; otherwise, $P_{Pre_i} = 1$. In the latter case, the action's success probability P_{a_i} is determined by uncertainties and given by the conditional success probability $P_{uc_i} = P(a_i \ succeed \mid Pre_i \ satisfied)$, which reflects the likelihood of a_i succeeding once its preconditions are met.

Initialization of action's conditional successful probability. The initial value of P_{uc_i} is assigned according to its corresponding action a_i's type and available evidence. For *attacker actions*, we adopt a worst-case assumption that the action is repeated until success, yielding $P_{uc_i}^{attacker} = 1$. *User action* success probabilities are influenced by uncertain human factors, such as security awareness, knowledge, experience, organizational culture, or emotional state at time T_i. These factors are partially observable at the population level, public threat landscape reports provide statistical data on user behavior for some scenarios.

Example 7.2. The public available average click-through rate on malicious links reported by security vendors can serve as a practical estimate for profiling $P_{uc_i}^{user}$ for actions like "user-clicks-malicious-redirect-url-in-message".

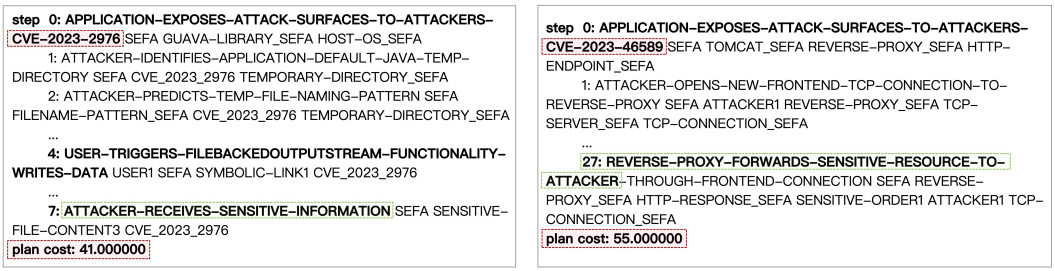
We assign initial probabilities via semantic approximation. For those lacking explicit statistics, we conservatively set $P_{uc_i}^{user} = 1$. At runtime, $P_{uc_i}^{user}$ is updated using newly retrieved statistics. For *application* and *infrastructure actions*, whose uncertainties currently lack beliefs, we conservatively set $P_{uc_i}^{app} = 1$ and $P_{uc_i}^{infra} = 1$, maximizing P_{a_i} and the overall risk score RS_{AP} to avoid overlooking high-impact vulnerabilities. Future work will refine belief models for different action types.

Encoding of action success probabilities in AED/AEDIs. In practice, we use the A* epsilon search algorithm implemented in Metric-FF 2.0 [19] to generate risk-maximizing APs. This requires specifying $\mathcal{M}$ as `(:metric minimize (total-cost))` across the six AEDIs, and assigning each action a_i a cost by appending `(increase (total-cost) <int>)` to its effect Eff_i in the AED domain file. To ensure equivalence between minimizing total cost and maximizing $RS_{AP} = \left(\prod_{i=1}^{n} P_{a_i}\right) \cdot S_v = \left(\prod_{i=1}^{n} (P_{Pre_i} \cdot P_{uc_i})\right) \cdot S_v$, we (a) transform the product-maximization objective into a planner-compatible sum-minimization metric, and (b) decompose RS_{AP} into per-action costs to align with the planner's selection logic. The planner naturally skips actions with $P_{Pre_i} = 0$, and when $P_{Pre_i} = 1$, selects actions based on their costs. We therefore define each action cost using a negative logarithm transformation. Let $\mathcal{A}^{EXP} \subseteq \mathcal{A}$ denotes the set of all the *application vulnerability exposure actions*. The cost of a_i is:

$$\text{cost}(a_i) = \begin{cases} \max\big(1,\ -100 \ln(\max(\delta, 0.1 \cdot S_v \cdot P_{uc_i}))\big), & \text{if } a_i \in \mathcal{A}^{EXP}, \\ \max\big(1,\ -100 \ln(\max(\delta, P_{uc_i}))\big). \end{cases} \tag{3}$$

where $\delta = 10^{-6}$ avoids $\ln(0)$, 0.1 scales the vulnerability severity S_v (CVSS v3.0) of the CVE (corresponding to an exposure action) into $[0, 1]$. At runtime, if P_{uc_i} changes based on the latest

observations, the corresponding action cost in the AED domain file is updated, enabling the planner to continuously select the AP that maximizes RS_{AP}.



(a) T_1 (b) T_2

Fig. 6. The SEFA's risk-maximum *AP* targeting Information disclosure after the user accepts security education.

Example 7.3. At timeslot T_1, SEFA exposes two vulnerabilities: CVE-2023-2976 and CVE-2023-46589. As shown in Figure 6 (a), Metric-FF generates a maximum-risk AP that targets Information disclosure exploits CVE-2023-2976 with a total cost of 41. At T_2, following user security education, the conditional success probability P_{uc} of a necessary step ("user-triggers-filebackedoutputstream-functionality-writes-data") for exploiting CVE-2023-2976 significantly decreases, making CVE-2023-2976 unlikely to be successfully exploited and thus substantially reducing its actual risk. Consequently, after updating the corresponding action cost in AED^{T_i} via Eq. (3), the planner generates a new risk-maximizing path exploiting CVE-2023-46589 for the same goal. This path has a higher cost of 55, indicating lower risk (Figure 6 (b)). Comparing the attack paths before and after the change also reveals the step-wise exploitation process in an interpretable manner.

Computation of overall risk score. After obtaining the six attack paths *AP*s that pose the highest S, T, R, I, D, E threats and their individual risk scores, the overall risk is calculated as:

$$RS = \sum_{g \in \{S,T,R,I,D,E\}} w_g \cdot RS_{AP_g} \tag{4}$$

where w_g denotes the weight of each threat category. As microservices interact, computing the *RS* of an implementation without considering the implementations of other microservices composing the application may misrepresent the overall risk, as even modern containerization techniques cannot fully eliminate infection chains [46]. SAQURE therefore computes each implementation's *RS* by combining the implementation of other microservice as a whole application. The overall EADSA process is summarized in Figure 7 and Algorithm 1. At runtime, when adaptation is triggered, the Analyzer uses EADSA to compute *RS* for the current implementation *cur* and all new candidate implementations *new*s using the latest AED^{T_i} and corresponding $AEDI_g^{T_i}$, $g \in \{S, T, R, I, D, E\}$.

7.3 Security and Qos-driven Self-adaptation (R2.B)

Upon being invoked by the Analyzer, the Planner retrieves the up-to-date estimates of availability, response time, and risk score for the current implementation *cur* and new implementations *new*s to evaluate the instant cost brought by applying a new implementation ($cur \in news$) to the managed application. The goal is to minimize the security risk while maintaining acceptable availability and response time. We follow the cost formulation introduced in AQUA [3] and denote it as the *Instant Cost*, which is defined as follows:

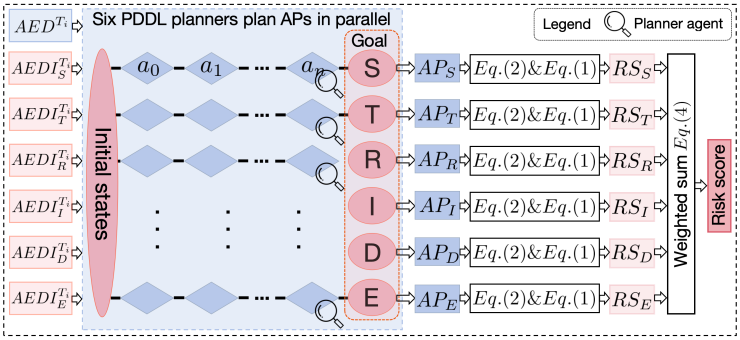


Fig. 7. The workflow of EADSA.

Algorithm 1: EADSA

Input: Latest AED: $AED^{T_i} = (\mathcal{T}, \mathcal{P}, \mathcal{F}, \mathcal{A})$;
Latest six AEDIs of the implementation: $AEDI_g^{T_i}$s, where $AEDI_g^{T_i} = (\mathcal{D}, \mathcal{O}, \mathcal{IS}, \mathcal{GS}, \mathcal{M})$, $g \in \{S, T, R, I, D, E\}$;
Severity and probability lookup file: f_p.
Output: Risk score of the implementation: RS.
`// Step 1: Plan six risk-maximize APs in parallel`
Run six Metric-FF planner agents in parallel, each independently solving its corresponding $p_{AP_g} = \{\text{AED}^{T_i}, \text{AEDI}_g^{T_i}\}$ to generate the maximum-risk AP_g for $g \in \{S, T, R, I, D, E\}$.
`// Step 2: Compute the risk score`
for *each* $AP_g \in \{AP_S, AP_T, AP_R, AP_I, AP_D, AP_E\}$ **do**
  Retrieve the severity of the exploited vulnerability S_v from f_p.
  for *each action* a_i *in* $AP_g = \langle a_0, a_1, \ldots, a_i \rangle$ **do**
    Retrieve the probability P_{Pre_i} of action a_i from f_p.
    Retrieve the probability P_{uc_i} of action a_i from f_p.
    Compute the success probability P_{a_i} of action a_i using Eq. (2).
  end
  Compute the risk score RS_{AP_g} of AP_g using Eq. (1).
end
Compute the overall risk score RS of the implementation using Eq. (4).
return RS.

$$IC_{new_i} = \frac{\omega_a \cdot A_{new_i} + \omega_t \cdot (1 - T_{new_i}) + \omega_r \cdot (1 - RS_{new_i})}{\omega_a \cdot A_{cur} + \omega_t \cdot (1 - T_{cur}) + \omega_r \cdot (1 - RS_{cur})} \tag{5}$$

where A_{cur}, T_{cur}, and RS_{cur} denote the standardized[2] and normalized[3] availability, response time, and risk score of the current implementation. Similarly, A_{new_i}, T_{new_i}, and RS_{new_i} refer to those of

[2]Standardization removes the influence of different scales among availability, response time, and security risk score.
[3]Normalization maps these metrics into the [0.1, 0.9] range.

the new_i implementation. $\omega_a, \omega_t, \omega_r \in [0, 1]$, $\sum_{i \in \{a,t,r\}} \omega_i = 1$, are importance weights[4]. All metrics are evaluated at runtime, with security risk scored by the Analyzer's EADSA.

To prevent the Planner from immediately selecting an implementation that fails to ensure long-term operation, SAQURE follows AQUA [3]: it evaluates the long-term impact of each candidate by running PRISM probabilistic model checker [26] on its DTMC model to simulate its operation over an infinite horizon. PRISM enumerates all execution paths from the `START` state to the `SUCCESS` state. Each path is a sequence of states, each representing a microservice implementation. An instant cost for each state is computed using Eq. (5). PRISM accumulates instant costs of states along the path to a cumulative cost. The probability of each path is the product of its state transition probabilities. The expected long-term cumulative cost is obtained by summing the product of each path's cumulative cost and probability. The Planner then selects the implementation with the lowest cumulative cost, records it in the Knowledge module, and invokes the Executer to apply it.

8 Evaluation

This section presents the research questions (Section 8.1), offline specification of the DTMC model (Section 8.2) and AED/AEDIs (Section 8.3), runtime management implementation (Section 8.4), and the experimental design (Section 8.5). The replication package is available at https://github.com/SuperCorleone/SAQURE.

8.1 Research Questions

Our objective is to evaluate whether SAQURE can select the optimal adaptation option that minimizes security risk while maintaining acceptable QoS, under both static and dynamic runtime conditions, and within strict real-time constraints. Additionally, we assess the impact of explicitly accounting for uncertainty in security assessment and examine SAQURE's runtime scalability. Accordingly, we address the following research questions:

RQ1: What is the effectiveness of SAQURE in selecting the optimal adaptation option compared to baseline approaches and ablation variants when runtime conditions remain unchanged?

RQ2: What is the effectiveness of SAQURE in selecting the optimal adaptation option under changing runtime conditions, compared to AQUA?

RQ3: What is the effect of explicitly accounting for uncertainty in dynamic security assessment on SAQURE's selection of the optimal adaptation option?

RQ4: What is the scalability of SAQURE with respect the number of exposed vulnerabilities and candidate implementations?

8.2 DTMC of SEFA and TeaStore

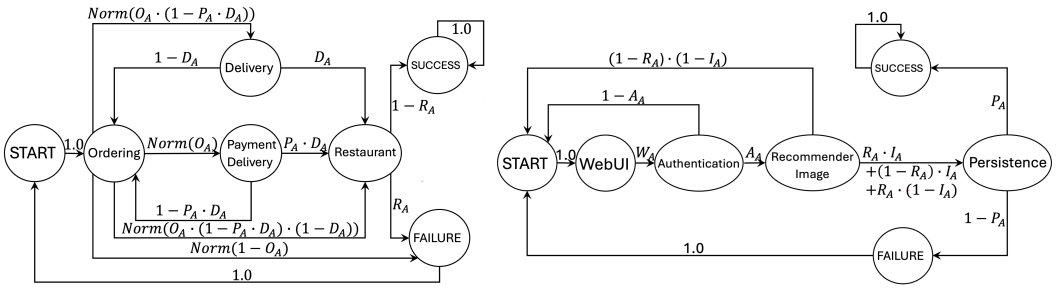
Following the approach in AQUA [3], we construct the initial DTMC of the managed microservice application. The DTMC is mechanically derived from application data and interaction flows. States represent (i) execution entry points, (ii) individual microservices, and (iii) terminal outcomes (SUCCESS or FAILURE). Transitions capture inter-service dependencies: a transition exists if a source microservice invokes an API of a target microservice. Each DTMC is persisted in the Knowledge module, and queried by the PRISM probabilistic model checker at runtime to simulate managed application's execution and evaluate long-term cumulative costs of adaptation options.

Figure 8 (a) shows the DTMC specified for SEFA ($Norm(\cdot)$ denotes the normalization operation.), comprising seven states: one initial state (`START`), two absorbing states (`SUCCESS`, `FAILURE`), and four intermediate states corresponding to `ordering`, `delivery`, `restaurant`, and a composite

[4]We set $\omega_a = 0.3$, $\omega_t = 0.3$, and $\omega_r = 0.4$ to prioritize security while maintaining acceptable availability and response time.

payment+delivery state capturing the joint outcome of concurrent payment and delivery invocations. Transition probabilities are parameterized by service availabilities. In the modeled workflow, ordering first invokes payment and delivery concurrently; if both succeed, execution proceeds to restaurant. Upon failure, ordering retries with delivery only (cash-on-delivery), and if this also fails, it directly invokes restaurant for a take-away option. The restaurant then either confirms the order (SUCCESS) or, if unavailable, FAILURE and restarts the process, yielding the absorption semantics captured by the DTMC.

Apart from SEFA, we evaluate SAQURE using TeaStore, a benchmark microservice-based application implemented in Java EE. TeaStore provides a simplified online store via six independent microservices: WebUI (front-end), Authentication (user login and sessions), Persistence (database access), Recommender (product recommendations), Image Provider (product images), and Registry (service registration and discovery). Figure 8 (b) shows the DTMC model for TeaStore, comprising 7 states: one initial (START), two absorbing (SUCCESS, FAILURE), and four intermediate states corresponding to WebUI, Authentication, Persistence, and a composite Recommender+Image Provider state for parallel invocation. The Registry service is excluded as it does not directly affect user-perceived functionality and can be reflected implicitly in the availability parameters. From the user's perspective, access to TeaStore starts via the WebUI service, which first invokes Authentication to verify identity. Upon successful authentication, Recommender and Image Provider are queried in parallel for personalized recommendations and product images. If at least one responds successfully, the process proceeds to the Persistence to complete the data operations for browsing or ordering. Failures in any critical service return the process to START, while successful completion of all required services confirms the user request and reaches SUCCESS. Otherwise, retries are triggered from START.



(a) SEFA DTMC model. (b) TeaStore DTMC model.

Fig. 8. The DTMC models of SEFA and TeaStore.

8.3 Offline AED & AEDIs specification

The initial AED and AEDI specifications were generated offline. A small subset of vulnerabilities was specified by experts, while the majority were generated automatically using our LLM-based method, CVE2AP [9]. The generations were evaluated using the three-layer S3Eval pipeline [10], followed by expert review and post-processing to ensure consistency with the AED.

Vulnerability list. Specifically, reports produced by the OWASP Dependency-Check Gradle plugin [36] for SEFA [3] and TeaStore [45] were used as seed data and supplemented with the NVD/CVE entries [30], yielding a candidate vulnerability set. After filtering by the criteria in Table 6 and verifying their attack path specifications (detailed below), 100 vulnerabilities (Critical:

38; High: 52; Medium: 10) were retained for the AED. Table 7 lists five representative vulnerabilities, the complete list is available in the replication package.

Table 6. Selection Criteria for Microservice-related Vulnerabilities

No.	Criterion Description
1	Published after 2022; CVSS score $\geq$ 4.0
2	Relevant to common microservice dependencies, communication protocols, etc. (e.g., Spring)
3	Exposure depends on vulnerable technical compositions, insecure permissions/access paths, or mis-configured security controls
4	Exploitation feasible without chaining to other vulnerabilities
5	Exploitation can be achieved using typical resources, without requiring privileged access, prior system knowledge, or elaborate social engineering.

Table 7. Representative Vulnerabilities for Offline AED Specification

Vubernability	CVSS	Affected Component
CVE-2025-22228	7.4	Spring Security (5.7.0-5.7.15, 5.8.0-5.8.17, 6.0.0-6.0.15, 6.1.0-6.1.13, 6.2.0-6.2.9, 6.3.0-6.3.7, 6.4.0-6.4.3)
CVE-2025-24813	9.8	Apache Tomcat (9.0.0-9.0.98, 10.1.0-10.1.34, 11.0.0-11.0.2)
CVE-2024-22243	8.1	Spring Framework (5.3.0-5.3.31, 6.0.0-6.0.16, 6.1.0-6.1.3)
CVE-2024-34447	7.5	Bouncy Castle ($\leq$ 1.78)
CVE-2023-34055	6.5	Spring Boot (2.7.0-2.7.17, 3.0.0-3.0.12, 3.1.0-3.1.5)

AED. For expert-specified vulnerabilities (21), we conducted a systematic analysis from three perspectives: *Vulnerability characteristics*, including CVE descriptions, CWE categories, and known TTPs (tactics, techniques, and procedures); *Exploitation conditions*, including the inferred entry point and required system state (covering vulnerable technical composition, weak permissions & access paths, and missing or misconfigured security mechanisms); *Environmental context*, including exogenous entities involved in exploitation (e.g., attacker hosts, representative users, reverse proxies, and DNS infrastructure). This analysis was complemented by (i) the configuration of the managed microservice applications (e.g., dependency manifests, service communication topology, and security control settings), and (ii) snapshots of the simulated runtime environment at selected time points, with typical user behaviors from testbed logs (May.–Aug. 2025).

Based on these evidence and the adversary model (Section 4), each vulnerability was mapped to a set of actions in $\mathcal{A}$, encoded in the `(:action)` construct of AED^{T_i}. These actions span the four categories defined in AED, *application*, *attacker*, *user*, and *infrastructure* actions. Among these, each CVE has a dedicated *application vulnerability exposure action* a_i, named "`application-exposes-attack-surfaces-to-attackers-<CVE-ID>`". Such an action becomes applicable when its Pre_i are satisfied by the grounded system snapshot represented in the corresponding $AEDI^{T_i}$s (Section 6.2), thereby enabling subsequent exploitation actions to be considered during planning. Using this action-centric representation, we completed $\mathcal{T}$, $\mathcal{P}$, $\mathcal{F}$ using the `(:types)`, `(:predicates)`, and `(:functions)` constructs respectively. The expert-specified portion contributes 279 actions (124 application, 105 attacker, 35 user, and 15 infrastructure).

The remaining majority were automatically generated by CVE2AP [9]. CVE2AP generates PDDL-encoded attack path specifications from CVE descriptions. We apply its best-performing configuration: `GPT-5.5` [34] with two-shot prompting, an output template, and automated error feedback, with each vulnerability requiring an average of about 80 s to generate, as reported by CVE2AP. From the generated candidates, we retained 79 generations that passed all three S3Eval [10] verification layers. These were expert-reviewed and post-processed to align with AED conventions

(e.g., assigning action costs), adding 1,055 actions (630 application, 346 attacker, 19 user, and 60 infrastructure). The offline AED specification time is given in Section 9.4.

AEDIs. Each initial $AEDI^{T_i}$ instance was specified offline based on a concrete snapshot of a managed microservice application. Object instances were derived from application configurations, environment information, and candidate vulnerabilities, and defined in $\mathcal{O}$, encoded in the `(:objects)` construct of $AEDI^{T_i}$ (e.g., `SEFA - application`, `user1 - user`). Instance states, relations, and attributes were initialized in $\mathcal{I}$ as predicates and function assignments, specified in the `(:init)` construct (e.g., "`(has-library SEFA jettison-library_SEFA)`" and "`(= (version jettison-library_SEFA) 1005001001)`"). For each snapshot, six $AEDI^{T_i}$ problem files were generated, corresponding to the STRIDE threat categories (e.g., `spoofing.pddl`, `tampering.pddl`). These files share identical $\mathcal{O}$ and $\mathcal{I}$ but differ in their goals $\mathcal{G}$, specified in the `(:goal)` construct. All AEDIs used the same metric $\mathcal{M}$, specified as `(:metric minimize (total-cost))`, with `total-cost` initialized to 0 in the `(:init)` construct via `(= (total-cost) 0)`.

The initial AED domain and six AEDI problem files are maintained in the Knowledge module. At runtime, changes observed by the Monitor are periodically reflected in the AED^{T_i} or synchronized across $AEDI^{T_i}$s via the MAPE-K loop. The complete AED and AEDIs are publicly available in the accompanying repository.

8.4 Runtime Management Implementation

SAQURE builds on AQUA [3]. The Analyzer integrates the EADSA method (Section 7.2) and updates the AED domain file and AEDI problem files during execution simulation via file-level operations according to identified changes. The Plan module selects the optimal adaptation option using the PRISM probabilistic model checker [26]. Specifically, after constructing the initial DTMC offline, PRISM models for the current (`cur`) and candidate (`new`) configurations are automatically generated using PRISM modeling language. PRISM is then used to simulate the managed application's execution and compute the expected long-term cost over an infinite horizon using the property `R{"cost"}=? [ C ]`, where `C` denotes total accumulated cost along the infinite execution path. The expected cost is computed for the current configuration and all available new adaptation options, the option with the lowest value is forwarded to the Execute component for adaptation.

8.5 Design of the Evaluation

To address RQ1–RQ4, we designed a series of controlled experiments simulating adaptation scenarios on two benchmark systems, SEFA and TeaStore.

Controlled variables. The controlled variables relevant to RQ1–RQ4 are listed in Table 8.

Compared methods. We repeatedly simulate the execution of the managed applications under SAQURE and a set of compared methods to evaluate SAQURE's effectiveness and key design choices. The compared methods include baseline approaches and ablation variants. The baseline methods considered in our experiments are as follows:

- **RAMSES [41]**, which selects adaptation decisions based on a utility function that considers only QoS factors, namely availability and response time.
- **AQUA [3]**, which extends RAMSES's QoS-based adaptation by additionally incorporating a security value that is computed offline and reused at runtime.

RAMSES and AQUA are selected as baselines because they represent the most closely related self-adaptive frameworks for microservice applications: RAMSES provides a QoS-only baseline that SAQURE extends, while AQUA is one representative approach that integrates security assessment into the MAPE-K adaptation loop for microservices's security. To assess the impact of specific design choices in SAQURE, we construct following ablation variants:

- **SAQURE$^{-}_{para}$**, in which the Analyzer employs a single planner to sequentially generate six attack paths (each targeting one STRIDE goal), instead of invoking six planners in parallel as in SAQURE. This variant isolates the effect of parallel AP planning.
- **SAQURE$^{-}_{\text{dy}P_{a_i}}$**, in which the conditional success probability is fixed to 1 ($P_{uc_i} = 1$), such that action success probability $P_{a_i} = P_{Pre_i} \times P_{uc_i} = 1$, whenever the precondition is satisfied ($P_{Pre_i} = 1$). In another words, all actions in the planned APs are assumed to succeed with 100% probability. This variant isolates the impact of quantifying uncertainty.

Measurements. For each experiment, we record the **risk score** computed by each method and the resulting **adaptation decision (AD)** it produces. In addition, we collect the following *runtime* metrics during the simulated execution of the MAPE-K loop:

- **Long-term cost (unitless)**. The expected long-term cumulative cost associated with the implementation option selected by a method. Lower values indicate better adaptation choices.
- **Runtime decision time (ms)**. The wall-clock time required by the Plan module to produce an adaptation decision. Lower values are better.
- **Runtime risk assessment time (ms)**. The wall-clock time taken by the Analyzer module to compute the risk score for a given implementation. Lower values are better.

For the offline phase, we additionally measure the **configuration time (ms)**, i.e., the wall-clock time required to prepare knowledge artifacts prior to runtime. For SAQURE, offline configuration time comprises initial *AED/AEDI specification time* and *initial DTMC generation time*.

Unless stated otherwise, each experiment is repeated 10 times, and we report aggregated statistics (mean, Q1, and Q3) for time metrics.

Adaptation objective. Our adaptation objective is to minimize security risk while maintaining acceptable QoS (availability and response time) through self-adaptation.

Table 8 maps each research question to the systems as well as controlled factors of interest, compared methods, and measurements in our controlled simulation experiments. We run all the experiments on a MacBook Pro (2023) equipped with an Apple M2 Pro chip (12-core CPU, 19-core GPU), 16 GB LPDDR5 RAM, a 512 GB SSD, running macOS Sonoma 14.3.1, and Java RE 17.0.10. In addition, we set weights $w_g = 1, g \in \{S, T, R, I, D, E\}$ in Eq. (4) to ensure that each STRIDE category contributes equally to the final risk score *RS*. In the following, we discuss the most relevant results.

Table 8. Map of RQs to Systems, Controlled Variables, Compared Methods and Measurements

RQ	Systems	Controlled variables	Compared methods	Measurements
RQ1	SEFA TeaStore	1. Configuration of the managed system (deployed microservices, available implementations, etc.) 2. State of the environment (attackers, users, infrastructures) 3. QoS level, and vulnerability of the implementation	1. RAMSES 2. AQUA 3. SAQURE$^{-}_{para}$	1. Long-term cost 2. Runtime decision time 3. Adaptation decision
RQ2	SEFA TeaStore	1. Configuration of the managed system (deployed microservices, available implementations, etc.) 2. State of the environment (attackers, users, infrastructures) 3. QoS level, and vulnerability of the implementation	AQUA	1. Long-term cost 2. Runtime decision time 3. Risk score 4. Adaptation decision
RQ3	SEFA	1. Configuration of the managed system (deployed microservices, available implementations, etc.) 2. State of the environment (attackers, users, infrastructures) 3. QoS level, and vulnerability of the implementation	SAQURE$^{-}_{\text{dy}P_{a_i}}$	1. Long-term cost 2. Runtime decision time 3. Risk score 4. Adaptation decision
RQ4	SEFA	1. Number of vulnerabilities in the system 2. Number of available implementations 3. Size of the managed system	AQUA	1. Runtime decision time 2. Runtime risk assessment time 3. Configuration time

9 Experimental Results

This section presents the experimental results and analysis for RQ1–RQ4 (Sections 9.1–9.4).

9.1 RQ1: Effectiveness without Runtime Change

Each scenario starts from a distinct initial configuration at time T_1, with varying estimates of availability, response time, and exposed vulnerabilities for the composed microservice implementations. For the target microservice, the current implementation may remain unchanged, and two or three alternative implementations are available for adaptation. Tables 9 and 10 summarize the operating scenarios and available implementation options for SEFA (TeaStore settings are provided in the replication package). In Table 10, grey cells indicate improvement over the current implementation, while underlined cells denote degradation. A brief description of each scenario follows.

- **s_1:** Two options, one slightly improves availability and response time while reducing vulnerability by exposing the system to fewer or less severe vulnerabilities (lower CVSS scores); the other improves availability and response time but increases vulnerability.
- **s_2:** Two options, each optimizing metrics to different degrees.
- **s_3:** Three options for a critical microservice, each improving one metric while keeping the others.
- **s_4:** Three options for a non-critical microservice, each improving one metric while leaving other metrics unchanged.
- **s_5:** Three options for a non-critical microservice, each improving one metric but significantly degrading other metrics.

Additionally, in our experiment, for AQUA [3], which relies on static CVSS scores computed offline to estimate security within the adaptation loop, the vulnerability score of each microservice is computed as the sum of its exposed vulnerabilities' CVSS scores. Each scenario undergoes execution until a full adaptation loop completes. Other settings follow those described in the original papers.

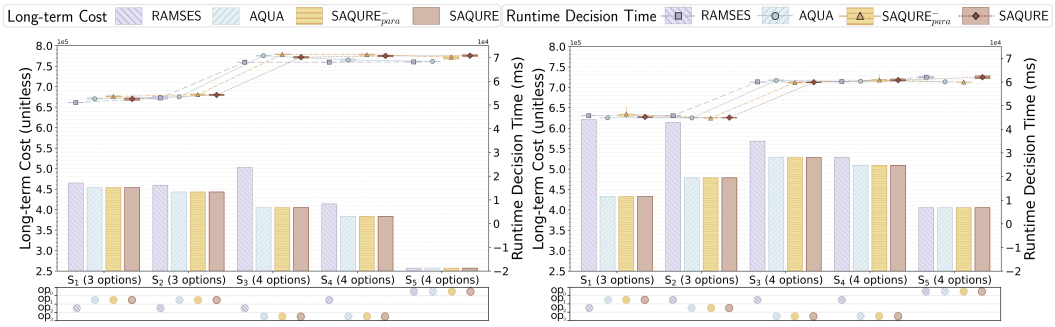
Table 9. Five Scenarios' Configurations of SEFA by Differentiating Configurations at T_1

Scenario	Microservice implementation	Estimates at T_1							
		Availability	Response time	Vulnerabilities					
s_1 & s_2 & s_3	Restaurant_0	0.9	50	CVE-2024-38286	CVE-2024-12798	CVE-2024-38820	CVE-2025-22228	CVE-2023-44487	
	Ordering_0	0.92	350	CVE-2024-38286	CVE-2024-22243	CVE-2024-34447	CVE-2023-6378		
	Delivery proxy_0	0.85	300	CVE-2024-38286	CVE-2024-38820	CVE-2022-40150	CVE-2024-38816	CVE-2022-1471	
	Payment proxy_0	0.9	450	CVE-2024-38286	CVE-2023-34055	CVE-2024-34447			
s_4	Restaurant_0	0.55	500	CVE-2024-38286	CVE-2024-12798	CVE-2024-38820	CVE-2025-22228	CVE-2023-44487	
	Ordering_0	0.8	300	CVE-2024-38286	CVE-2024-22243	CVE-2024-34447	CVE-2023-6378	CVE-2025-24813	
	Delivery proxy_0	0.85	300	CVE-2024-38286	CVE-2024-38820	CVE-2022-40150	CVE-2024-38816	CVE-2022-1471	
	Payment proxy_0	0.9	450	CVE-2024-38286	CVE-2023-34055	CVE-2024-34447			
s_5	Restaurant_0	0.9	50	CVE-2024-38286	CVE-2024-12798	CVE-2023-44487			
	Ordering_0	0.92	350	CVE-2024-38286	CVE-2024-22243	CVE-2024-34447	CVE-2023-6378		
	Delivery proxy_0	0.85	300	CVE-2024-38286	CVE-2024-38820	CVE-2022-40150	CVE-2024-38816	CVE-2022-1471	
	Payment proxy_0	0.9	450	CVE-2024-38286	CVE-2023-34055	CVE-2024-34447			

Table 10. Available Implementation Options at T_1 for Different Scenarios of SEFA

Scenario / Microservice	Implementation option	Estimates at T_1							
		Availability	Response time	Vulnerabilities					
s_1 / Restaurant	Restaurant_1	0.93	30	CVE-2024-38286	CVE-2024-12798	CVE-2024-38820			
	Restaurant_2	0.96	10	CVE-2024-38286	CVE-2024-12798	CVE-2024-38820	CVE-2025-22228	CVE-2023-44487	CVE-2025-24813
s_2 / Restaurant	Restaurant_1	0.95	40	CVE-2024-38286	CVE-2024-38820				
	Restaurant_2	0.98	10	CVE-2024-38286	CVE-2024-12798	CVE-2025-24813			
s_3 / Ordering	Ordering_1	0.88	350	CVE-2024-38286	CVE-2024-22243	CVE-2024-34447	CVE-2023-6378	CVE-2025-24813	
	Ordering_2	0.8	250	CVE-2024-38286	CVE-2024-22243	CVE-2024-34447	CVE-2023-6378	CVE-2025-24813	
	Ordering_3	0.8	350	CVE-2024-38286	CVE-2024-22243	CVE-2023-34055			
s_4 / Delivery proxy	Delivery proxy_1	0.95	300	CVE-2024-38286	CVE-2024-38820	CVE-2022-40150	CVE-2024-38816	CVE-2022-1471	
	Delivery proxy_2	0.85	200	CVE-2024-38286	CVE-2024-38820	CVE-2022-40150	CVE-2024-38816	CVE-2022-1471	
	Delivery proxy_3	0.85	300	CVE-2024-38286	CVE-2024-38820	CVE-2022-40150			
s_5 / Payment proxy	Payment proxy_1	0.92	500	CVE-2024-38286	CVE-2023-34055	CVE-2024-34447	CVE-2022-1471	CVE-2025-24813	
	Payment proxy_2	0.7	300	CVE-2024-38286	CVE-2023-34055	CVE-2024-34447	CVE-2022-1471	CVE-2025-24813	
	Payment proxy_3	0.7	500	CVE-2024-38286	CVE-2023-34055				

Case Study 1 (SEFA). Figure 9 (a) shows the long-term costs (bars, left y-axis) and runtime decision times (box plots with Q1–mean–Q3 computed from 10 identical runs, right y-axis, with



(a) SEFA. (b) TeaStore.

Fig. 9. Comparison of method effectiveness without runtime changes in two case studies.

mean connecting lines) for the compared methods and SAQURE across scenarios s_1–s_5 (x-axis, annotated with the total number of cur and new options). Below the x-axis, a table indicates the option selected by each method in each scenario. In scenario s_1, RAMSES selects Restaurant$_2$, yielding the highest long-term cost ($\approx 4.65 \times 10^5$). Although Restaurant$_2$ provides slightly greater QoS improvements (an additional 0.03 in availability and 20 ms in response time improvement compared to Restaurant$_1$), these benefits are relatively minor. Moreover, its higher vulnerability increases security risk, resulting in a larger long-term cost, making it a worse decision than Restaurant$_1$. AQUA, SAQURE$^{-}_{para}$, and SAQURE are capable of selecting the option with the smallest long-term cost (4.54×10^5), i.e., Restaurant$_1$. For the remaining scenarios s_2–s_4, a similar pattern can be observed: methods that account for vulnerability consistently achieve lower long-term costs and select the optimal option. In s_5, all methods retain the current implementation Restaurant$_0$. This is because each new option improves one metric but significantly degrades the others. The holistic evaluations of AQUA, SAQURE$^{-}_{para}$, and SAQURE, which account for both QoS and vulnerability, thus indicate that no new implementation outperforms the current one.

Regarding runtime overhead, the box plots of SAQURE$^{-}_{para}$ and SAQURE remain at nearly the same level as those of RAMSES and AQUA. This indicates that, although SAQURE$^{-}_{para}$ and SAQURE introduce dynamic security assessment during runtime, it introduces negligible additional runtime. The AP planning and EADSA can be executed in near real time. However, runtime decision time grows with the number of implementation options: in s_3–s_5 (four options), the runtime decision time increases by $\approx$ 2–4 s compared to s_1–s_2 (three options). This is because each additional option requires PRISM to perform extra probability check during execution simulation.

Case Study 2 (TeaStore). For TeaStore, as shown in Figure 9 (b), the pattern is same as SEFA: AQUA, SAQURE$^{-}_{para}$, and SAQURE consistently select the optimal options with the lowest long-term costs. Runtime decision times of SAQURE$^{-}_{para}$ and SAQURE remain nearly identical to those of RAMSES and AQUA, indicating that dynamic security assessment introduces negligible overhead. Although SAQURE performs AP planning parallelly (versus sequentially in SAQURE$^{-}_{para}$), the difference in runtime remains negligible. In the setup involving six AP planning problems p_{AP}s, each task completes almost in real time, making sequential or parallel execution practically equivalent.

RQ1 Summary: Across all static scenarios without runtime changes, SAQURE, which performs EADSA at runtime, consistently reduces the long-term cost compared to RAMSES. Additionally, EADSA introduces negligible overhead, with its runtime decision time nearly identical to

RAMSES's. Overall, without the runtime changes, SAQURE effectively selects the optimal option that minimizes security risk while maintaining acceptable QoS.

9.2 RQ2: Effectiveness under Runtime Change

To address RQ2, we evaluate SAQURE under runtime changes and compare it with AQUA across three scenarios on SEFA and TeaStore. As in RQ1, each scenario starts from a distinct initial configuration and offers multiple implementations for adapting a single microservice. Unlike RQ1, RQ2 introduces runtime changes at time T_2 that alter the security posture of available implementations. These changes, including updates to security controls, dependency versions, and action success probabilities, are reflected through corresponding updates to the AED domain or AEDI problem specifications. To evaluate SAQURE's ability to dynamically assess security risk and select adaptation options under evolving environments while controlling for other factors, we directly simulate runtime changes by applying predefined file-level updates to the corresponding AED or AEDI specifications during the execution simulation. Additionally, all implementations share the same availability and response time as the current implementation, remaining constant throughout execution, while their vulnerability profiles are changed. Tables 11 and 12 respectively summarize implementation options for the three operating scenarios in SEFA and TeaStore, reporting vulnerability estimates before (T_1) and after (T_2) the runtime change. Grey cells indicate vulnerabilities that become newly exploitable or cease to be exploitable at T_2 compared with T_1 due to runtime changes. The three operating scenarios for SEFA and TeaStore are available in our replication package.

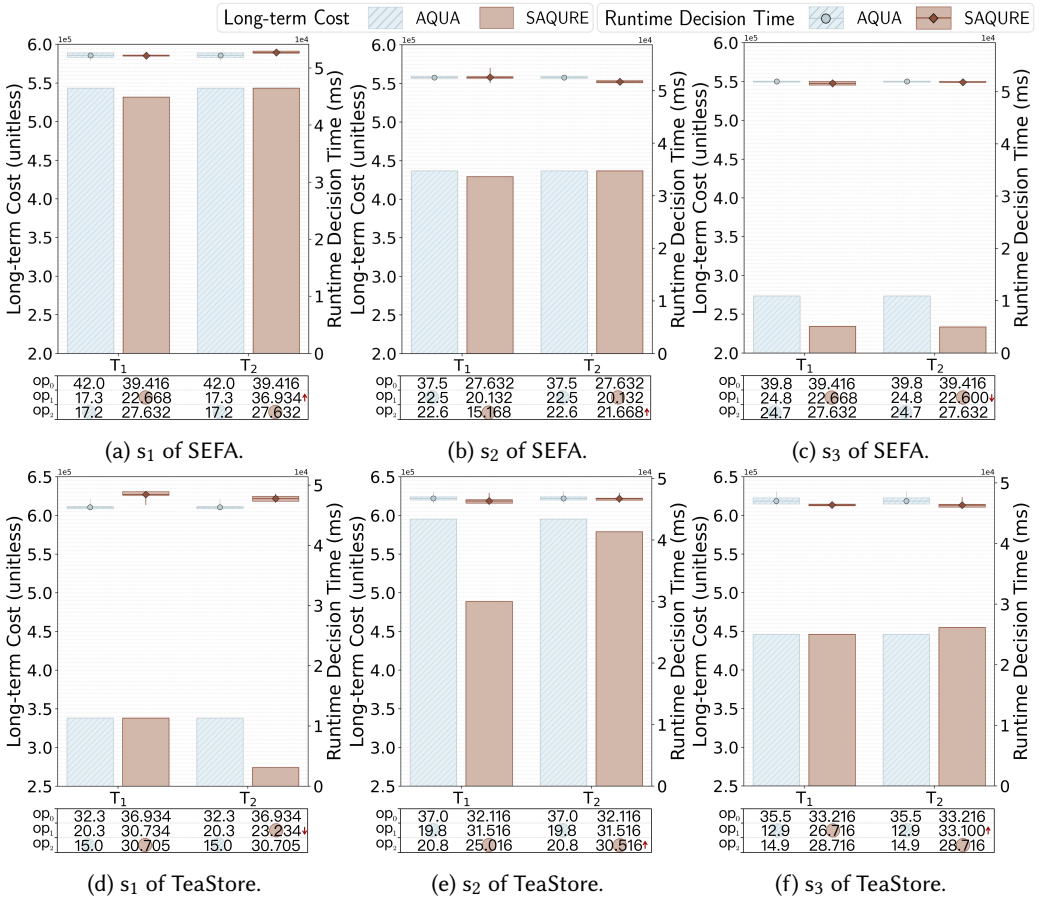
Table 11. Available Implementation Options with Runtime Changes at T_2 for Different Scenarios of SEFA

Scenario / Microservice	Microservice implementation	Vulnerabilities at T_1			Change at T_2	Vulnerabilities at T_2			
s_1 / Restaurant	Restaurant_1	CVE-2022-1471	CVE-2025-22228		Restaurant_1's WAF for blocking malicious requests shut down.	CVE-2022-1471	CVE-2025-22228	CVE-2025-24813	
	Restaurant_2	CVE-2022-1471	CVE-2024-34447			CVE-2022-1471	CVE-2024-34447		
s_2 / Payment proxy	Payment proxy_1	CVE-2022-1471	CVE-2024-38820	CVE-2024-34447	The version of Payment proxy_2's Springboot: 2.7.18.1 → 2.7.10.1.	CVE-2022-1471	CVE-2024-38820	CVE-2024-34447	
	Payment proxy_2	CVE-2022-1471	CVE-2024-38820	CVE-2025-22228		CVE-2022-1471	CVE-2024-38820	CVE-2025-22228	CVE-2023-34055
s_3 / Ordering	Ordering_1	CVE-2022-1471	CVE-2024-38286	CVE-2024-34447	P_{uc} of "user-inputs-sensitive- information-in-the-webpage": 0.005 → 0	CVE-2022-1471	CVE-2024-38286		
	Ordering_2	CVE-2022-1471	CVE-2024-38286	CVE-2025-22228		CVE-2022-1471	CVE-2024-38286	CVE-2025-22228	

Table 12. Available Implementation Options with Runtime Changes at T_2 for Different Scenarios of TeaStore

Scenario / Microservice	Microservice implementation	Vulnerabilities at T_1			Change at T_2	Vulnerabilities at T_2			
s_1 / WebUI	WebUI_1	CVE-2024-38809	CVE-2024-34447	CVE-2024-47072	The version of WebUI_1's XStream: 1.4.10.9 → 1.4.21	CVE-2024-38809	CVE-2024-34447		
	WebUI_2	CVE-2024-38286	CVE-2024-38816			CVE-2024-38286	CVE-2024-38816		
s_2 / Authentication	Authentication_1	CVE-2024-12798	CVE-2025-22228	CVE-2023-34055	Authentication_2's sanity check for asn1 is closed or doesn't work	CVE-2024-12798	CVE-2025-22228	CVE-2023-34055	
	Authentication_2	CVE-2024-12798	CVE-2025-22228	CVE-2023-46589		CVE-2024-12798	CVE-2025-22228	CVE-2023-46589	CVE-2023-33202
s_3 / Persistence	Persistence_1	CVE-2025-22228	CVE-2023-33202		The version of Persistence_1's Guava: 31.1.1.0 → 31.1.0.0	CVE-2025-22228	CVE-2023-33202	CVE-2023-2976	
	Persistence_2	CVE-2025-22228	CVE-2024-47072			CVE-2025-22228	CVE-2024-47072		

Case Study 1 (SEFA). Figure 10 (a)–(c) compares AQUA and SAQURE across s_1–s_3 of SEFA with different runtime changes. The x-axis distinguishes T_1 (before the change) and T_2 (after the change). The left y-axis shows long-term cost, and the right y-axis (with Q1–mean–Q3 from 10 identical runs, with mean connecting lines) indicates runtime decision time. Beneath the x-axis, a table reports each option's risk score at T_1 and T_2, where arrows (↑, ↓) denote increases or decreases in risk from T_1 to T_2, and circles mark the option selected by each method. In scenario s_1, at T_1, based on the summed CVSS scores of each option, Restaurant_1 carries slightly higher risk than Restaurant_2, so AQUA selects Restaurant_2, yielding a higher long-term cost (Figure 10(a)). At T_2, disabling Restaurant_1's WAF malicious-request blocking exposes this implementation to an additional critical vulnerability CVE-2025-24813 (9.8). AQUA, relying on offline precomputed fixed risk scores, continues selecting Restaurant_2. In contrast, SAQURE incorporates this runtime change by updating Restaurant_1's AEDI (commenting the "`WAF-malicious-request-block`" entry), and



(a) s_1 of SEFA. (b) s_2 of SEFA. (c) s_3 of SEFA.

(d) s_1 of TeaStore. (e) s_2 of TeaStore. (f) s_3 of TeaStore.

Fig. 10. Comparison of the effectiveness of AQUA, and SAQURE with runtime changes at T_2.

re-planning APs, and accordingly updating the risk score of Restaurant$_1$ via EADSA (22.668 → 36.934), resulting in a switch to the option of Restaurant$_2$, and achieving the lowest long-term cost.

In scenario s_2, at T_1, *Payment_proxy*$_2$ exhibits higher CVSS sum, thus AQUA selects *Payment_proxy*$_1$, whereas SAQURE selects *Payment_proxy*$_2$, achieving a lower long-term cost. At T_2, a runtime library version change for *Payment_proxy*$_2$ introduces a new vulnerability CVE-2023-34055 (CVSS 6.5, impacts Spring Boot versions 2.7.0-2.7.17, 3.0.0-3.0.12, and 3.1.0-3.1.5). AQUA, incapable of dynamic assessment at the runtime, keeps its risk estimations and selection, with a higher long-term cost. SAQURE, however, reflects the runtime version change by updating Payment proxy$_2$'s AEDIs (update <$N_{version}$> of "`(= (version springboot_SEFA) <`$N_{version}$`>)`" from "`2007018001`" to "`2007010001`") and re-evaluate based on the updated AED/AEDI. This enables SAQURE to successfully update *Payment_proxy*$_2$'s risk score (15.168 → 21.668), and switch to Payment proxy$_1$, achieving the lowest long-term cost.

In scenario s_3, a critical runtime change occurs at T_2: widespread public security education reduces the conditional success probability of the action "`user-inputs-sensitive-information-in-the-webpage`" from 0.005 to 0, rendering CVE-2024-34447, which depends on this action to proceed along its AP, becomes non-exploitable. SAQURE captures this change, updates the risk score of

Ordering_proxy$_1$ accordingly, and selects the optimal option with the lowest long-term cost. This adjustment is enabled by EADSA, which defined an action's success probability P_a as the product of the deterministic precondition satisfaction probability ($Ppre$) and the conditional success probability (P_{uc}). Whenever either factor changes, EADSA updates P_a and reflects this change by modifying the corresponding action cost (the `<int>` value in "`:effect (and ··· (increase (total-cost) <int>))`") in the AED, and re-evaluate the risk using the updated AED.

By leveraging updatable AED/AEDIs, as well as EADSA to plan APs based on the latest AED/AEDI, compute risk scores that jointly consider both vulnerability severity and step-wise exploitation success probability (accounting for deterministic and uncertain factors), SAQURE effectively selects options with the lowest risk and acceptable QoS under dynamic runtime conditions. This EADSA introduces negligible overhead: SAQURE's box plot aligns closely with or even falls below AQUA's.

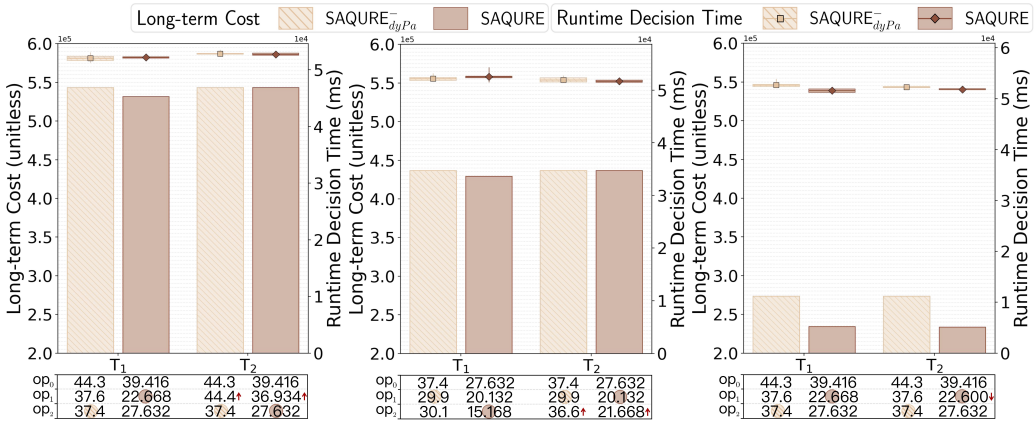
Case Study 2 (TeaStore). Figure 10 (d)–(f) shows the results for TeaStore scenarios. Across s_1–s_3, the same pattern emerges: SAQURE consistently outperforms AQUA by selecting the option with the lowest long-term cost while dynamically updating risk scores under diverse runtime changes. In s_1, the external library XStream used by $WebUI_1$ is upgraded from 1.4.10.9 to 1.4.21 at T_2, eliminating CVE-2024-47072 (7.5) and resulting in $WebUI_1$ being exposed to less severe vulnerabilities at T_2. In s_3, the Guava library used by $Persistence_1$ is downgraded from 31.1.1.0 to 31.1.0.0 at T_2, thereby introducing an additional exploitable vulnerability, CVE-2023-2976 (7.1). SAQURE successfully reflects the version change across the AEDIs by updating the `<int>` value of the corresponding `(= (version <external_lib_TeaStore>) <int>)` entry and re-running EADSA, decreasing the risk score of $WebUI_1$ in s_1 from 30.734 to 23.234, while increasing that of $Persistence_1$ in s_3 from 26.716 to 33.100. In s_1, disabling the ASN.1 sanity check exposes $Authentication_2$ to CVE-2023-33202 (5.5) at T_2. SAQURE synchronizes this change via updating Authentication$_2$'s AEDIs (e.g., commenting out the "`(endpoint-has-asn1-sanity-checks TeaStore pem-endpoint_TeaStore)`") and correspondingly increases its risk score from 25.016 to 30.516 via EADSA. Additionally, SAQURE maintains a runtime decision time nearly identical to AQUA's. For adaptations with three options and up to four vulnerabilities each, EADSA satisfies stringent runtime requirements.

RQ2 Summary: Under evolving environments, SAQURE effectively selects the optimal option that minimizes security risk while preserving acceptable QoS, introducing negligible overhead.

9.3 RQ3: The Effect of Accounting for Uncertainty

To answer RQ3, we compare SAQURE with its ablation variant SAQURE$^{-}_{\mathrm{dy}P_{a_i}}$, which disregards uncertainty by assuming a 100% step-wise success probability, and reuse part of the experimental setup from RQ2, including SEFA scenarios s_1–s_3. Across s_1–s_3 at T_1, both adaptation options expose the same number of vulnerabilities but differ in one critical CVE: CVE-2025-22228 (CVSS 7.4) versus CVE-2024-34447 (CVSS 7.5), each contributing to the highest-risk spoofing (AP_S) and information disclosure (AP_I) attack paths. The remaining highest-risk attack paths (AP_T, AP_R, AP_D, AP_E) rely on other identical vulnerabilities present in both options. Scenarios s_1–s_3 retain their respective runtime changes at T_2: s_1, s_2 respectively involves a deterministic change of security controls and component's version, while s_3 involves an uncertainty-driven change caused by human factors.

Result. As shown in Figure 11 (a)–(c), across s_1–s_3 at T_1, SAQURE$^{-}_{\mathrm{dy}Pa_i}$, which eliminates the impact of uncertainty by assuming 100% step-wise successful probability, selects the option exposed to CVE-2025-22228, yielding higher long-term costs. In contrast, SAQURE selects the option exposed to the higher-CVSS CVE-2024-34447, yet achieves the lowest long-term cost. This result stems from the fact that in realistic environments, uncertain factors prevent attackers from guaranteeing the success of every attack step. For CVE-2024-34447, the successful probability of one required



(a) s_1 (deterministic change). (b) s_2 (deterministic change). (c) s_3 (uncertainty-driven change).

Fig. 11. Comparison of SAQURE$^{-}_{\mathrm{dy}P_{a_i}}$ and SAQURE for optimal adaptation option selection.

step, "USER-INPUTS-SENSITIVE-INFORMATION-IN-THE-WEBPAGE", is greatly degraded and fluctuated by uncertain human factors. SAQURE's EADSA accounts for this by computing each AP's risk as the product of vulnerability severity and the step-wise success probability (Eqs. (1)), where step success is jointly determined by deterministic preconditions P_{pre} and uncertain P_{uc} factors (Eq. (2)). For CVE-2024-34447, SAQURE estimates $P_{uc} = 0.005$ at T_1 based on threat intelligence, thereby substantially reducing the realized risk of CVE-2024-34447. This also explains why, in s_3, as shown in Figure 11(c), when P_{uc} for this step decreases to 0 due to public security education at T_2, only SAQURE reduces the risk of $Ordering_1$. In contrast, although SAQURE$^{-}_{\mathrm{dy}P_{a_i}}$ updates the risk score to reflect the deterministic changes across AEDI in s_1, s_2, it fails to capture this uncertainty-driven change in s_3. Overall, quantifying uncertainty in security assessment is critical for selecting the optimal adaptation option and adapting to uncertainty-driven changes.

RQ3 Summary: SAQURE quantifies uncertainty in security assessment by explicitly accounting for both deterministic and uncertain exploitation factors in action successful probability. This is critical for selecting the optimal adaptation option and adapting to uncertainty-driven changes.

9.4 RQ4: Scalability

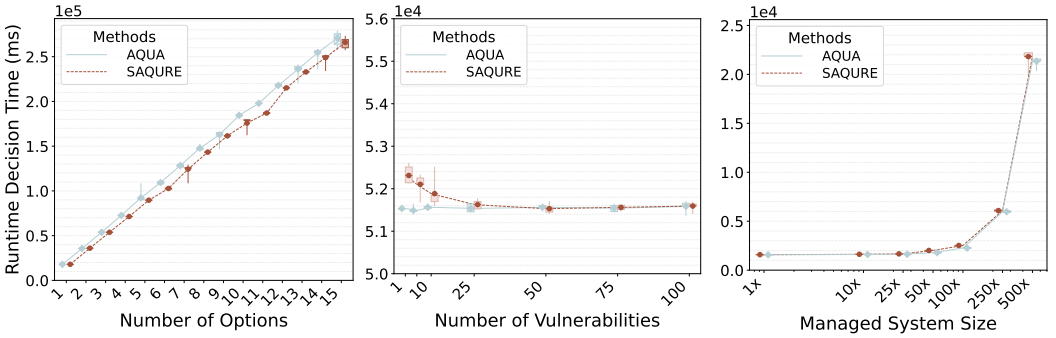
To answer RQ4, we formulate three sub-questions, each evaluated using case study 1 (SEFA). Unless otherwise specified, each experiment varies one factor at a time while keeping other factors fixed.

(1). *How does **runtime decision time** vary with **(i)** the number of available implementation options, **(ii)** the number of vulnerabilities, and **(iii)** the size of the managed system?* We conduct three experiments. For (i), we vary the number of available implementations for the *payment proxy* microservice from 1 to 15, with the managed-system size fixed at 1× and three vulnerabilities per implementation. For (ii), we vary the number of vulnerabilities per implementation for the *restaurant* microservice from 1 to 100, with two available alternatives. As SEFA comprises four microservices, the total number of vulnerabilities ranges from 4 to 400. For (iii), following AQUA [3], we scale SEFA's DTMC model from 1× to 500×, corresponding to 4–2,000 states, while fixing three vulnerabilities per implementation. In all experiments, we compare SAQURE and AQUA with fiixed availability, response-time estimates, and vulnerability assignments.

(2). *How does **runtime risk assessment time** change with **(i)** the number of vulnerabilities and **(ii)** the size of the managed system?* We reuse the settings of **RQ4(1)(ii)–(iii)** and measure EADSA execution time for computing the risk score of $restaurant_0$. Only SAQURE is evaluated.

(3). *How does **configuration time** scale with **(i)** the number of vulnerabilities and **(ii)** the size of the managed system?* We reuse the settings of **RQ4(1)(ii)–(iii)** for both factors.

Scalability of Runtime Decision Time. Figure 12(a) shows how runtime decision time changes as *the number of available implementation* options for *Payment* increases from 1 to 15. Both methods exhibit roughly linear growth, from ~ 20 ms to ~ 280 ms. This results from PRISM performing an additional cost computation for each adaptation option, including DTMC model generation from its estimates (availability, response time, and vulnerability) and executing the model-checking run to calculate the cumulative long-term cost Each option thus adds roughly 14–20 s. Despite runtime EADSA, SAQURE's decision time is comparable to AQUA's, being roughly 0.1–10 s lower with > 2 options. Thus, SAQURE exhibits slightly better scalability in some cases.



(a) Vs. number of options. (b) Vs. number of vulnerabilities. (c) Vs. managed system size.

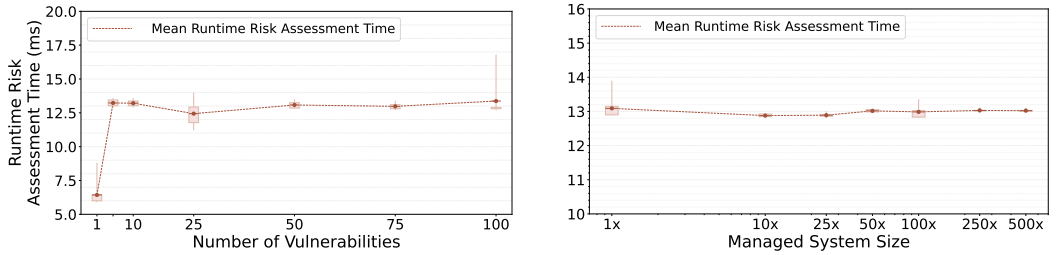
Fig. 12. The scalability of runtime decision time for AQUA and SAQURE.

Figure 12(b) shows how runtime decision time varies with *exposed vulnerabilities* for one microservice. Despite performing PDDL-based attack-path planning and risk estimation at runtime, SAQURE's decision times show low variance and remain comparable to or only slightly higher than AQUA's. SAQURE exhibits excellent scalability with respect to the number of vulnerabilities.

Figure 12(c) shows how runtime decision time scales with *the managed system size*. We make two observations: (i) AQUA and SAQURE show a similar trend in runtime decision time as the system size increases, as they use the same DTMC model and PRISM process; (ii) when the manged system size (i.e., the number of states in the corresponding DTMC model) is below 400 (100× SEFA), runtime decision time scales well with system size, while a dramatic increase from 2 s to 21.5 s occurs beyond 400 states. The PRISM's probability check dominates the runtime decision time.

Scalability of Runtime Risk Assessment Time. Figure 13(a) plots runtime risk-assessment time, e.g., the time required by EADSA to plan six APs in parallel and compute the risk score, against *the number of exposed vulnerabilities* in $restaurant_0$. The mean trend line indicates that, within the measured range (up to 100 vulnerabilities), SAQURE's runtime risk-assessment time remains essentially constant. EADSA's average time cost is below 17 ms, negligible compared with the runtime decision time.

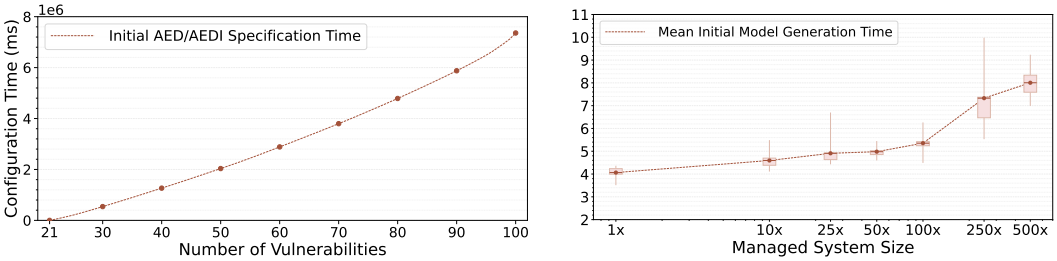
Figure 13(b) shows how runtime risk assessment time varies with *the managed system size*. It remains largely unaffected by system size, ranging from 225–275 ms.



(a) Vs. number of vulnerabilities. (b) Vs. managed system size.

Fig. 13. The scalability of runtime risk assessment time of SAQURE (EADSA).

Scalability of Configuration Time. Figure 14(a) shows how *vulnerability number* impacts initial AED/AEDI specification time. Starting from 21 expert-specified vulnerabilities, we automatically generated 79 additional vulnerabilities using CVE2AP [9], requiring approximately 93.2 s per vulnerability on average (about 2 h in total). The cumulative configuration time increases linearly, with a constant per-vulnerability marginal cost, demonstrating that SAQURE's offline AED specification scales efficiently as additional vulnerabilities are incorporated through CVE2AP.



(a) Vs. number of vulnerabilities. (b) Vs. managed system size.

Fig. 14. The scalability of offline configuration time of SAQURE.

Figure 14(b) shows how initial DTMC generation time varies with *the managed system size*. Model generation time increases slowly up to 400 states (100× SEFA) but rises more rapidly thereafter. Nevertheless, even at 2,000 states (500× SEFA), it takes only 8 ms in the worst case.

RQ4 Summary: SAQURE's runtime decision time shows limited scalability with respect to the number of options and managed system size, primarily because PRISM's model-checking dominates the computation. Nevertheless, SAQURE maintains excellent scalability across vulnerabilities, thanks to EADSA, whose per-assessment overhead remains below 17 ms, enabling timely adaptation under stringent runtime constraints.

10 Discussion

This section discusses SAQURE's key strengths (Section 10.1) and limitations (Section 10.2), and describes how threats to validity have been mitigated (Section 10.3).

10.1 Strengths

Our evaluation shows that SAQURE dynamically assesses runtime risk and consistently selects the lowest-risk option while maintaining acceptable QoS for managed microservice applications under stringent timing constraints. Based on our observations, we summarize its key strengths.

Efficient optimal option selection under dynamic microservice conditions. SAQURE efficiently selects the lowest-risk option while maintaining acceptable QoS in dynamic microservice environments. Beyond approaches that assume static settings, SAQURE remains effective under runtime changes by synchronizing endogenous configurations and exogenous environmental data into AED/AEDIs, invoking EADSA to plan APs, and assessing risk based on the updated AED/AEDIs. Overall, SAQURE could mitigate risks from rapidly shifting microservice attack surface.

Near-real-time, scalable dynamic security assessment: EADSA. SAQURE integrates EADSA, a near–real-time and scalable dynamic security assessment method. As shown in RQ4(2), even with up to 21 vulnerabilities, EADSA completes assessments within 0.26 s, adding negligible runtime overhead. Benefiting from the real-time planning nature of the AI planner, EADSA can accommodate extensions to AED/AEDI while maintaining millisecond-level latency for AP planning and risk assessment, enabling timely runtime decision making.

Reality- and uncertainty-aware risk assessment via EADSA. Unlike approaches such as AQUA that rely solely on vulnerability severity (e.g., CVSS), SAQURE's EADSA evaluates system risk by explicitly accounting for uncertainty in exploitability, estimating the actual likelihood that a vulnerability can exert its critical impact based on both deterministic and uncertain factors. In practice, some high-CVSS vulnerabilities require very strict conditions to be exploitable, yielding relatively low real-world risk, whereas some vulnerabilities labeled as low or medium severity by NVD are trivial to exploit, posing higher practical risk. As shown in RQ3, EADSA captures this realism by defining an attack path's risk RS_{AP} as the product of its overall success probability $\prod_{i=1}^{n} P_{a_i}$ and the exploited vulnerability severity S_v. Moreover, EADSA attributes each step's success not only to deterministic preconditions P_{Pre} and uncertain factors P_{uc}, thereby explicitly incorporating non-deterministic real-world influences into risk estimation. Future work will refine belief models to better quantify uncertainty for different action types.

A generalized AP planning domain for microservice apps: AED. The AED specification is primarily derived from NVD vulnerability entries and is applicable beyond the case studies presented here (SEFA and TeaStore). By instantiating AEDIs according to an application's configuration and deployment environment, AED can be used to automatically generate APs for arbitrary microservice applications. We are exploring methods to accelerate AED updates to keep pace with, or even exceed, the rate of new vulnerabilities published in the NVD. Future work will also integrate information-rich open-source threat-intelligence feeds to further enrich the AED domain.

Real-time AP generation with strong explainability. Explainability in self-adaptation is increasingly recognized as crucial [4]. SAQURE uses AI planners to generate APs in real time based on the latest AED/AEDIs. Two examples of APs are shown in Figure 6 of Section 7.2, demonstrating significantly higher explainability of SAQURE than traditional rule- or logic-based tools such as TVA [24], or MulVAL [35]. Unlike approaches that rely on predefined primitives or formal rule trees, which can be difficult for non-experts to interpret, SAQURE's APs are intuitive and interpretable. This allows stakeholders without security expertise to understand attack progression, thereby enhancing both the transparency and practical utility of security analysis.

Fully automated runtime option selection. The runtime stages, including Analyser's EADSA and Planner's long-term cost computation, are fully automated by our toolchain (either for testing purposes or after the deployment in production).

10.2 Limitations

We summarizes the main limitations revealed by our evaluation and practical experience as follows.

Inability to assess unknown vulnerabilities. SAQURE's AED/AEDI specifications primarily rely on vulnerabilities published in the NVD and cannot plan APs exploiting unknown vulnerabilities. Emerging threats, such as zero-day attacks and advanced persistent threats (APTs), exploit unrecorded vulnerabilities in unpredictable ways, posing significant risks. For microservice applications with frequent updates, this challenge is amplified. Future work will explore integrating diversified threat intelligence sources beyond NVD to partially mitigate this limitation, enabling AED to better reflect emerging threats and support more comprehensive AP planning.

Inability to assess combined vulnerability exploitation. The current AED/AEDI specification allows planning of APs limited to a single vulnerability per path. Consequently, SAQURE cannot assess chained exploitation scenarios, which are common in APT campaigns where multiple vulnerabilities are exploited sequentially or combined via logical "AND" relationships. Extending AED/AEDI to capture such inter-vulnerability dependencies is an important direction for future work.

Incomplete support for automated change detection. In our experiments, SAQURE assumes that runtime changes and their corresponding AED/AEDI update rules are already known. A complete pipeline for automatically detecting real-world changes through the Monitor and mapping them to corresponding AED/AEDI updates has not yet been implemented. Developing this automated Monitor-to-Analyzer pipeline for model synchronization remains an important future work.

10.3 Threats to Validity

Following the standard classification of validity threats [48], we examined our study in terms of construct, conclusion, internal, and external validity. The potential threats to our approach and evaluation, and the corresponding mitigation strategies, are discussed below.

Construct validity. Threats to construct validity may arise from several sources. First, SAQURE minimizes the risk of the microservice application by incorporating a self-adaptive MAPE-K loop for management. However, this self-adaptive management loop could potentially enlarge the attack surface of the holistic system, raising a construct validity concern regarding whether the risk estimations performed by EADSA accurately reflect the overall system security posture. We mitigate this threat by explicitly limiting our adversarial model to application-level attack surfaces, such that an attacker's compromise is possible only when the application exposes attack surfaces associated with known CVEs. Additionally, exploitation is feasible without chaining to other vulnerabilities, which eliminates the attack paths combining management-loop vulnerabilities. Under these limitations, the management layer introduces no additional externally reachable attack vectors, ensuring that the risk estimations remain a valid reflection of the managed application's security. Second, assumptions made during the modeling of the managed systems could affect the results. To mitigate this, we evaluated SAQURE on well-established case studies: SEFA [41], widely used in the SAS community [3], and TeaStore [45], a standard benchmark in microservice research. The finial potential threat stems from not directly using explicit security measures, such as attack surfaces, to quantify the reduction of risk achieved by our method. We address this by employing the comprehensive "cost" metric from AQUA [3], which accounts for both security and QoS factors. In our study, we emphasize security by assigning weights of 0.4 to the security risk factor and 0.3 each to availability and response time.

Conclusion validity. Conclusion validity may be affected by experimental-environment variations, such as fluctuations in CPU/GPU utilization affecting runtime decision or risk assessment times. To mitigate this, the test machine ran only target experiments, minimizing interference from

background processes. Each experiment was repeated 10 times under identical settings, and results were reported as Q1–mean–Q3, reducing outlier influence and providing a more reliable basis.

Internal validity. Threats to internal validity may arise from bias in establishing cause–effect relationships. To mitigate this, we conducted extensive simulation campaigns, controlling the key factors listed in Table 8 for each research question. This direct manipulation ensures consistent conditions across repeated runs and enables systematic assessment of causal effects. To support reproducibility, all experimental results and software tools are publicly available.

External validity. Threats to external validity may arise if the characteristics of the managed systems in our case study do not represent those of other systems. We mitigate this by using SEFA, built on a modern stack with Spring and Docker, and TeaStore, a widely used Java-based microservice benchmark with REST APIs, both representative of contemporary microservices research. Although our experiments are conducted on a small set of microservices, we diversified operating scenarios to improve generalizability. SEFA and TeaStore exhibit similar adaptation patterns but differ in application domain, scale, service interaction topology, and runtime DTMC structure. Consistent results across these differences provide supporting evidence for the robustness of our findings. Larger and more varied systems could further strengthen external validity. Another potential threat arises if the managed applications expose more vulnerabilities or offer more implementation options. To address this, we conducted scalability studies measuring runtime decision time while varying the number of options from 1 to 15 and vulnerabilities in total for SEFA from 4 to 400, the size of the managed system from 1× to 500× SEFA (4 to 2,000 DTMC states), and evaluated runtime risk assessment time with respect to the number of exposed vulnerabilities. These experiments demonstrate the performance of our approach under different system conditions, supporting broader generalization of the results.

11 Conclusion

To mitigate the rapidly shifting attack surface and emerging threats in highly dynamic microservice-based applications, this paper presented SAQURE, an AI planning-driven self-adaptive system. SAQURE enables runtime adaptation by continuously synchronizing the endogenous configurations and exogenous environmental information into updatable AED/AEDIs, and by performing uncertainty-aware dynamic risk assessment through AI-planning-based EADSA to evaluate candidate adaptation options. Extensive experimental results on two representative microservice case studies demonstrate that SAQURE consistently identifies the option with the lowest security risk while preserving acceptable QoS under diverse runtime changes. The results further show that SAQURE operates in near real time and scales well with increasing vulnerabilities, making it suitable for highly dynamic microservice environments. Beyond the proposed system, this work releases a reusable and extensible generic AED for microservice applications. Overall, SAQURE takes a step toward practical, realistic, and uncertainty-aware runtime security management for modern microservice-based systems.

In the future, we plan to augment AED in two complementary ways: on one hand, to capture inter-vulnerability dependencies and enable the generation of chained exploitation paths; on the other hand, to enrich AED with multiple diversified threat intelligence sources beyond NVD, allowing it to capture unknown vulnerabilities. Moreover, we plan to further refine the belief models used in EADSA to better quantify uncertainty for different action types, thereby improving the fidelity of runtime risk assessment. We also plan to develop an automated Monitor-to-Analyzer pipeline for real-world change detection and AED/AEDI synchronization, and to investigate lightweight alternatives to PRISM for long-term cost estimation, such as simulation-based or analytical approximations, to further improve runtime scalability.

References

[1] 2025. OpenVAS — Open Vulnerability Assessment Scanner. https://www.openvas.org/. Accessed: 2025-12-04.

[2] Tanveer Hossain Bhuiyan, Apurba K Nandi, Hugh Medal, and Mahantesh Halappanavar. 2016. Minimizing expected maximum risk from cyber-attacks with probabilistic attack success. In *2016 IEEE Symposium on Technologies for Homeland Security (HST)*. IEEE, 1–6.

[3] Matteo Camilli, Fabio Luccioletti, Raffaela Mirandola, and Patrizia Scandurra. 2024. Integrated QoS-and Vulnerability-Driven Self-adaptation for Microservices Applications. In *International Conference on Service-Oriented Computing (ICSOC)*. Springer, 55–71.

[4] Matteo Camilli, Raffaela Mirandola, and Patrizia Scandurra. 2023. Enforcing resilience in cyber-physical systems via equilibrium verification at runtime. *ACM Transactions on Autonomous and Adaptive Systems* 18, 3 (2023), 1–32.

[5] Clinton Cao, Agathe Blaise, Sicco Verwer, and Filippo Rebecchi. 2022. Learning state machines to monitor and detect anomalies on a kubernetes cluster. In *Proceedings of the 17th International Conference on Availability, Reliability and Security*. 1–9.

[6] Pavlos Cheimonidis and Konstantinos Rantos. 2023. Dynamic risk assessment in cybersecurity: A systematic literature review. *Future Internet* 15, 10 (2023), 324.

[7] Pavlos Cheimonidis and Konstantinos Rantos. 2025. A novel proactive and dynamic cyber risk assessment methodology. *Computers Security* 154 (2025), 104439. doi:10.1016/j.cose.2025.104439

[8] Lin Cui and Raffaela Mirandola. 2025. RAdaptSQ: Real-Time AI-Planning and Environment-Aware Self-Adaptation to Optimize Security and QoS. *Softwaretechnik-Trends* 45, 4 (2025), 41–43. https://fb-swt.gi.de/fileadmin/FB/SWT/Softwaretechnik-Trends/Verzeichnis/Band_45_Heft_4/SSP25_paper_10.pdf

[9] Lin Cui, Vincenzo Scotti, and Raffaela Mirandola. 2026. CVE2AP: Automated Generation of PDDL-Encoded Attack Paths via Large Language Models. *arXiv preprint* (2026). To appear.

[10] Lin Cui, Vincenzo Scotti, and Raffaela Mirandola. 2026. S3Eval: A Three-Layer Pipeline for Automated Evaluation of PDDL-Encoded Attack Paths. In *Proceedings of the 4th International Workshop on Artificial Intelligence for Autonomous computing Systems (AI4AS), co-located with IEEE ACSOS 2026*. IEEE. To appear.

[11] Nicola Dragoni, Ivan Lanese, Stephan Thordal Larsen, Manuel Mazzara, Ruslan Mustafin, and Larisa Safina. 2018. Microservices: How to make your application scale. In *Perspectives of System Informatics: 11th International Andrei P. Ershov Informatics Conference, PSI 2017, Moscow, Russia, June 27-29, 2017, Revised Selected Papers 11*. Springer, 95–104.

[12] Ulfar Erlingsson and Fred B Schneider. 2000. IRM enforcement of Java stack inspection. In *Proceeding 2000 IEEE Symposium on Security and Privacy. S&P 2000*. IEEE, 246–255.

[13] FIRST CVSS Special Interest Group. 2019. Common Vulnerability Scoring System v3.1: Specification Document. https://www.first.org/cvss/v3-1/specification-document Version 3.1, Forum of Incident Response and Security Teams.

[14] Maria Fox and Derek Long. 2003. PDDL2. 1: An extension to PDDL for expressing temporal planning domains. *Journal of artificial intelligence research* 20 (2003), 61–124.

[15] Gartner Peer Community. 2023. Microservices Architecture: Have Engineering Organizations Found Success? https://www.gartner.com/peer-community/oneminuteinsights/omi-microservices-architecture-have-engineering-organizations-found-success-u6b

[16] Robin Gassais, Naser Ezzati-Jivan, Jose M Fernandez, Daniel Aloise, and Michel R Dagenais. 2020. Multi-level host-based intrusion detection system for Internet of things. *Journal of Cloud Computing* 9, 1 (2020), 62.

[17] Junjiang He, Tao Li, Beibei Li, Xiaolong Lan, Zhiyong Li, and Yunpeng Wang. 2021. An immune-based risk assessment method for digital virtual assets. *Computers & Security* 102 (2021), 102134.

[18] Victor Heorhiadi, Shriram Rajagopalan, Hani Jamjoom, Michael K Reiter, and Vyas Sekar. 2016. Gremlin: Systematic resilience testing of microservices. In *IEEE 36th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 57–66.

[19] Jörg Hoffmann. 2003. The Metric-FF Planning System: Translating"Ignoring Delete Lists"to Numeric State Variables. *Journal of artificial intelligence research* 20 (2003), 291–341.

[20] Eric M Hutchins, Michael J Cloppert, Rohan M Amin, et al. 2011. Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains. *Leading Issues in Information Warfare & Security Research* 1, 1 (2011), 80.

[21] Kyle Ingols, Richard Lippmann, and Keith Piwowarski. 2006. Practical attack graph generation for network defense. In *2006 22nd Annual computer security applications conference (ACSAC'06)*. IEEE, 121–130.

[22] Sushil Jajodia, Anup K Ghosh, Vipin Swarup, Cliff Wang, and X Sean Wang. 2011. *Moving target defense: creating asymmetric uncertainty for cyber threats*. Vol. 54. Springer Science & Business Media.

[23] Sushil Jajodia and Steven Noel. 2009. Topological vulnerability analysis: A powerful new approach for network attack prevention, detection, and response. In *Algorithms, architectures and information systems security*. World Scientific, 285–305.

[24] Sushil Jajodia, Steven Noel, and Brian O'berry. 2005. Topological analysis of network attack vulnerability. In *Managing Cyber Threats: Issues, Approaches, and Challenges*. Springer, 247–266.
[25] Hai Jin, Zhi Li, Deqing Zou, and Bin Yuan. 2019. Dseom: A framework for dynamic security evaluation and optimization of mtd in container-based cloud. *IEEE Transactions on Dependable and Secure Computing* 18, 3 (2019), 1125–1136.
[26] Marta Kwiatkowska, Gethin Norman, and David Parker. 2011. PRISM 4.0: Verification of probabilistic real-time systems. In *International conference on computer aided verification*. Springer, 585–591.
[27] Jan Lansky, Saqib Ali, Mokhtar Mohammadi, Mohammed Kamal Majeed, Sarkhel H Taher Karim, Shima Rashidi, Mehdi Hosseinzadeh, and Amir Masoud Rahmani. 2021. Deep learning-based intrusion detection systems: a systematic review. *IEEE Access* 9 (2021), 101574–101599.
[28] Yuanbo Li, Yuanmou Li, Guoqiang Wang, and Hongchao Hu. 2025. An Adaptive Dynamic Defense Strategy for Microservices Based on Deep Reinforcement Learning. *Electronics* 14, 20 (2025), 4096.
[29] Peter A Loscocco, Stephen D Smalley, Patrick A Muckelbauer, Ruth C Taylor, S Jeff Turner, and John F Farrell. 1998. The inevitability of failure: The flawed assumption of security in modern computing environments. In *In Proceedings of the 21st national information systems security conference*.
[30] National Institute of Standards and Technology. 2026. National Vulnerability Database (NVD). https://nvd.nist.gov/.
[31] Sergey Naumov and Ilya Kabanov. 2016. Dynamic framework for assessing cyber security risks in a changing environment. In *2016 International Conference on Information Science and Communications Technologies (ICISCT)*. IEEE, 1–4.
[32] J. R. Norris. 1998. *Markov Chains*. Cambridge University Press.
[33] National Institute of Standards and Technology. 2025. National Vulnerability Database (NVD). https://nvd.nist.gov. Accessed: 2025-07-25.
[34] OpenAI. 2026. Introducing GPT-5.5. https://openai.com/index/introducing-gpt-5-5/. Accessed: 2026-06-17.
[35] Xinming Ou, Sudhakar Govindavajhala, Andrew W Appel, et al. 2005. MulVAL: A logic-based network security analyzer.. In *USENIX security symposium*, Vol. 8. Baltimore, MD, 113–128.
[36] OWASP Foundation. 2025. OWASP Dependency-Check. https://owasp.org/www-project-dependency-check/. Accessed: 2025-07-09.
[37] OWASP Foundation. 2025. OWASP Top 10 Web Application Security Risks. https://owasp.org/Top10/. Accessed: 2025-20-12.
[38] Alessandro Palma, Claudio Cicimurri, and Marco Angelini. 2025. Progressive attack graph: a technique for scalable and adaptive attack graph generation: A. Palma et al. *International Journal of Information Security* 24, 5 (2025), 212.
[39] Gerald J Popek and Robert P Goldberg. 1974. Formal requirements for virtualizable third generation architectures. *Commun. ACM* 17, 7 (1974), 412–421.
[40] Vincenzo Riccio, Giancarlo Sorrentino, Matteo Camilli, Raffaela Mirandola, and Patrizia Scandurra. 2023. Engineering Self-adaptive Microservice Applications: An Experience Report. In *International Conference on Service-Oriented Computing*. Springer, 227–242.
[41] Vincenzo Riccio, Giancarlo Sorrentino, Ettore Zamponi, Matteo Camilli, Raffaela Mirandola, and Patrizia Scandurra. 2024. RAMSES: an Artifact Exemplar for Engineering Self-Adaptive Microservice Applications. In *Proceedings of the 19th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. 161–167.
[42] Adam Shostack. 2014. *Threat modeling: Designing for security*. John wiley & sons.
[43] Byungchul Tak, Canturk Isci, Sastry Duri, Nilton Bila, Shripad Nadgowda, and James Doran. 2017. Understanding security implications of using containers in the cloud. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*. 313–319.
[44] Tenable Network Security. 1998. Nessus Vulnerability Scanner. https://www.tenable.com/products/nessus. Accessed: 2025-02-10.
[45] Joakim Von Kistowski, Simon Eismann, Norbert Schmitt, André Bauer, Johannes Grohmann, and Samuel Kounev. 2018. Teastore: A micro-service reference application for benchmarking, modeling and resource management research. In *2018 IEEE 26th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*. IEEE, 223–236.
[46] Devi Priya VS, Sibi Chakkaravarthy Sethuraman, and Muhammad Khurram Khan. 2023. Container security: precaution levels, mitigation strategies, and research perspectives. *Computers & Security* 135 (2023), 103490.
[47] Danny Weyns. 2020. *An introduction to self-adaptive systems: A contemporary software engineering perspective*. John Wiley & Sons.
[48] Claes Wohlin, Per Runeson, Martin Höst, Magnus C Ohlsson, Björn Regnell, Anders Wesslén, et al. 2012. *Experimentation in software engineering*. Vol. 236. Springer.

Received xx xx 2026; revised xx xx 2026; accepted xx xx 2026